

DS5899: Special Topics in Data Science

Leveraging NLP in Asset Management

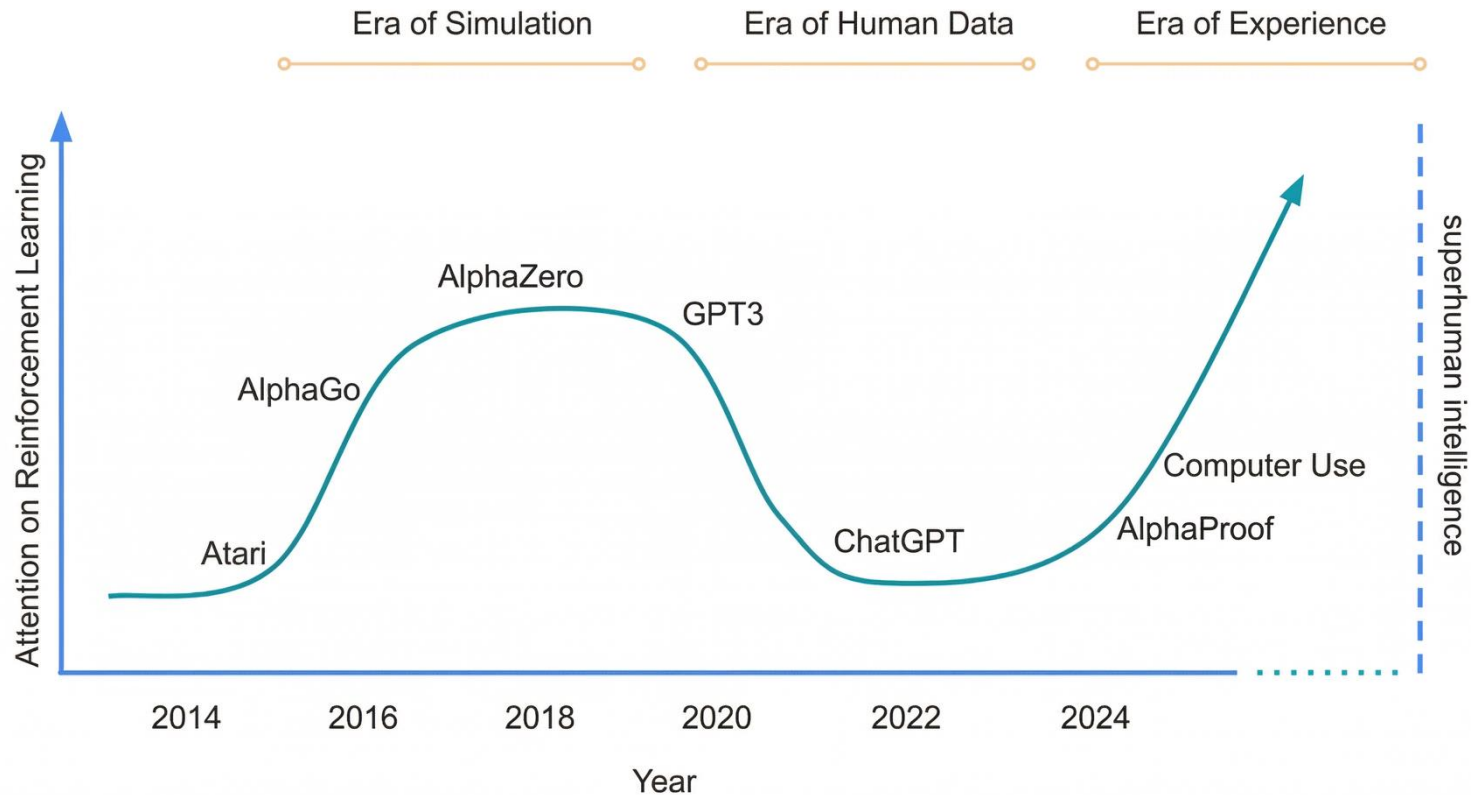
Week 12: Agent

Instructor: Che Guan, Ph.D.

Part 1:

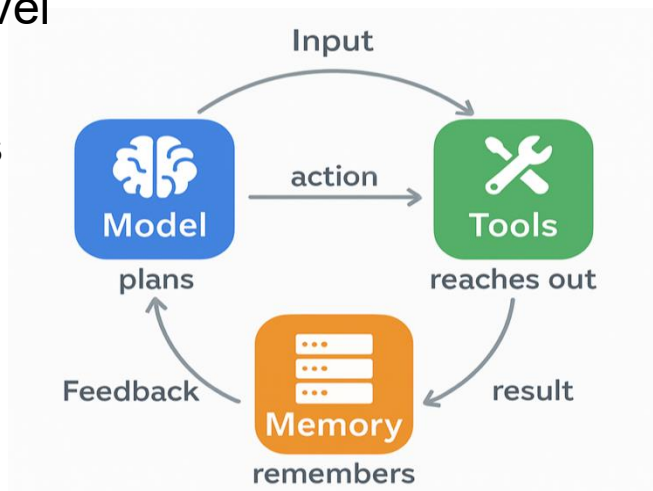
Agent

Welcome to Era of Experience



Agent Components

- An autonomous agent is typically composed of 4 main components:
 - **The Planner:** The LLM decomposes a high-level goal into smaller, actionable sub-tasks.
 - **Memory:** Short-term *and* Long-term Memories
 - **Tools:** The set of APIs or functions the agent can call.
 - **The Loop:** Unlike a single prompt, an agent runs in a loop: Observe → Think → Act → Observe.



ReAct

- A ReAct agent uses the “reasoning and acting” framework to combine CoT reasoning with external tool use.

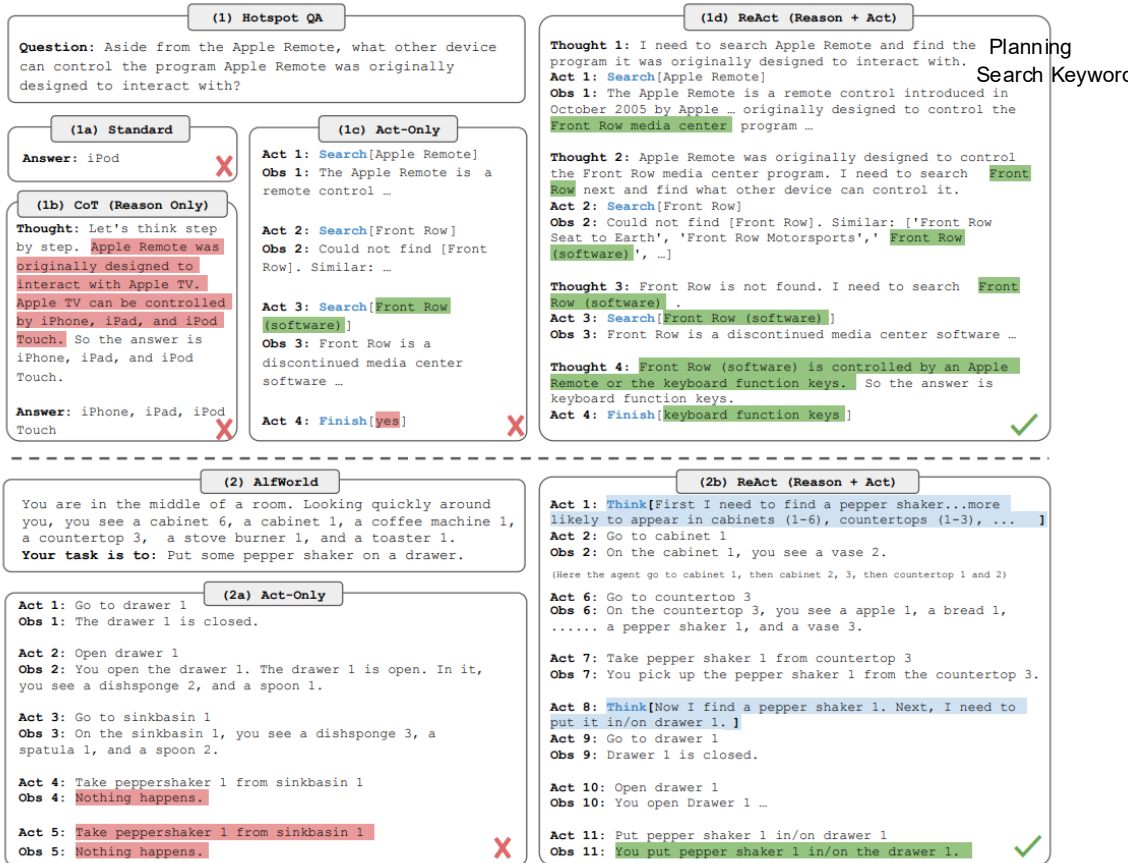
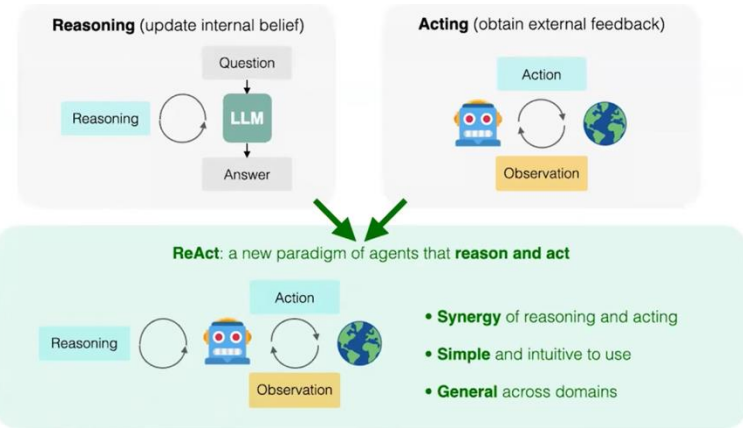
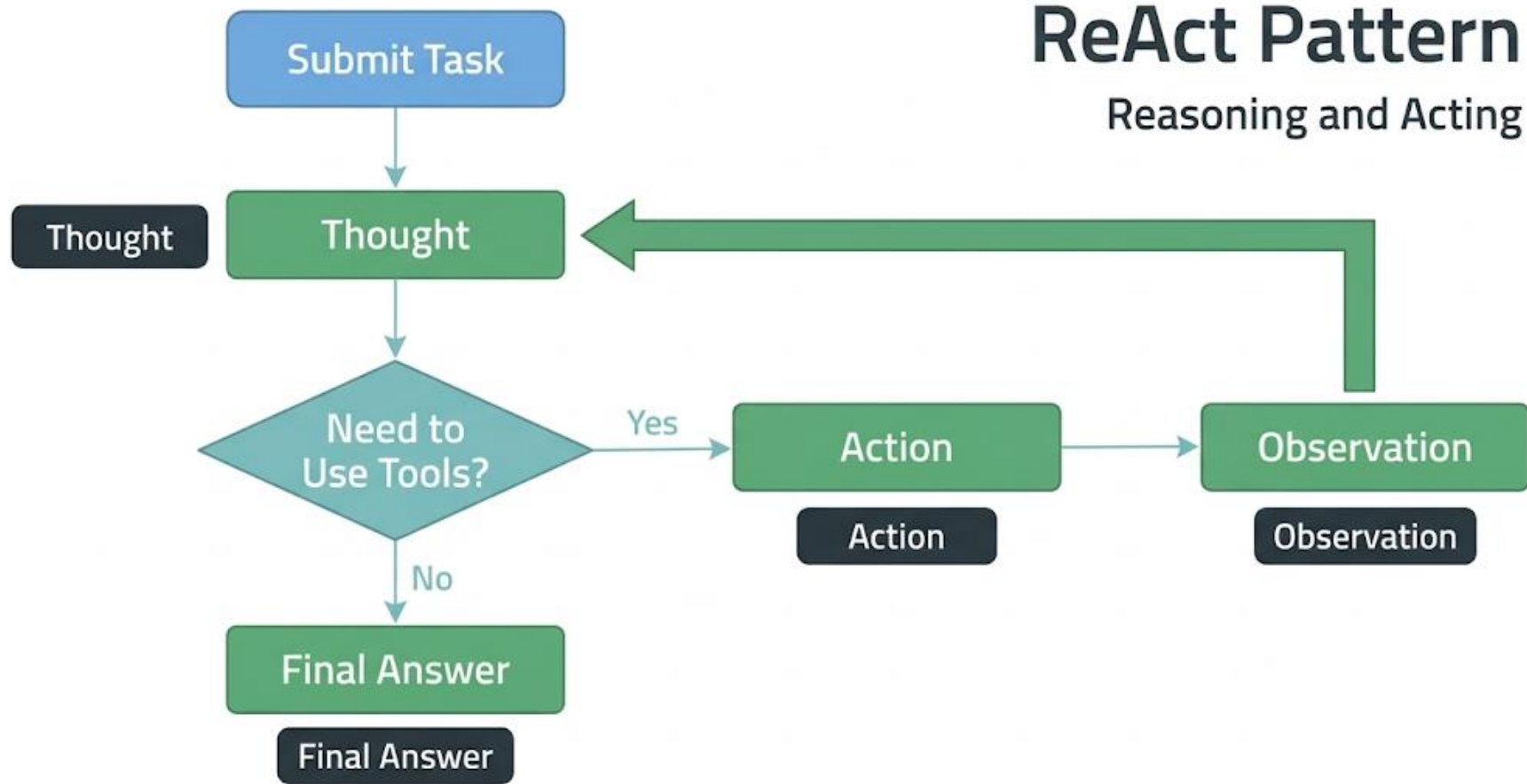


Figure 1: (1) Comparison of 4 prompting methods, (a) Standard, (b) Chain-of-thought (CoT, Reason Only), (c) Act-only, and (d) ReAct (Reason+Act), solving a HotspotQA (Yang et al., 2018) question. (2) Comparison of (a) Act-only and (b) ReAct prompting to solve an AlfWorld (Shridhar et al., 2020b) game. In both domains, we omit in-context examples in the prompt, and only show task solving trajectories generated by the model (Act, Thought) and the environment (Obs).

ReAct Pattern

Reasoning and Acting



A zero-shot ReAct Prompt

LLM

(GPT-4)

Thought: I need to find the current market capitalization of Apple, Nvidia, and Microsoft to understand how much it would cost to buy all of them.

Action: search["current market capitalization of Apple, Nvidia, and Microsoft"]

Observation: nothing found!

(My adversarial observation)

Thought: The search did not give any results. I will need to search for each company's market capitalization individually.

Action: search["current market capitalization of Apple"]

Observation: 'price': 180.75

(My adversarial observation)

Thought: This is the current price of Apple's stock, not its market capitalization. The market cap is calculated by multiplying the share price by the number of outstanding shares. Thus, I need to find the total number of Apple's outstanding shares to calculate its market cap.

Action: search["total number of Apple's outstanding shares"]



(Continues to solve the task)

Synergy: acting support reasoning, reasoning guides acting

ReAct System Prompt

Role Description

You are an AI assistant. Decompose tasks into steps. Use the following XML format for output at each step: `<task>` User task, `<thought>` Reasoning about what to do, `<action>` Call a tool. The execution result will be returned via `<observation>`. Repeat `thought/action`, until you have enough info to provide a `<final_answer>`. Answer Strict XML format must be used for all output steps.

- `<task>`: User-provided task
- `<thought>`: Reasoning
- `<action>`: Tool operation to be taken
- `<observation>`: Results returned by tools or environment
- `<final_answer>`: Final answer

Example 1



```
<task> Tell me how tall the Eiffel Tower is. </task>
<thought> I need to find the height of the Eiffel Tower. I can use the search tool. </thought>
<action> get_height("Eiffel Tower") </action>
<observation> The height of the Eiffel Tower is approximately 330 meters (including the antenna). </observation>
<thought> The search results show the height. I already have the answer. </thought>
<final_answer> The height of the Eiffel Tower is approximately 330 meters. </final_answer>
```

Available Tools

- `read_file(file_path)`: Read file content
- `write_file(filename, content)`: Write specified content to a file. Return 'Write Successful' on success.
- `run_terminal_command(command)`: Execute a terminal command

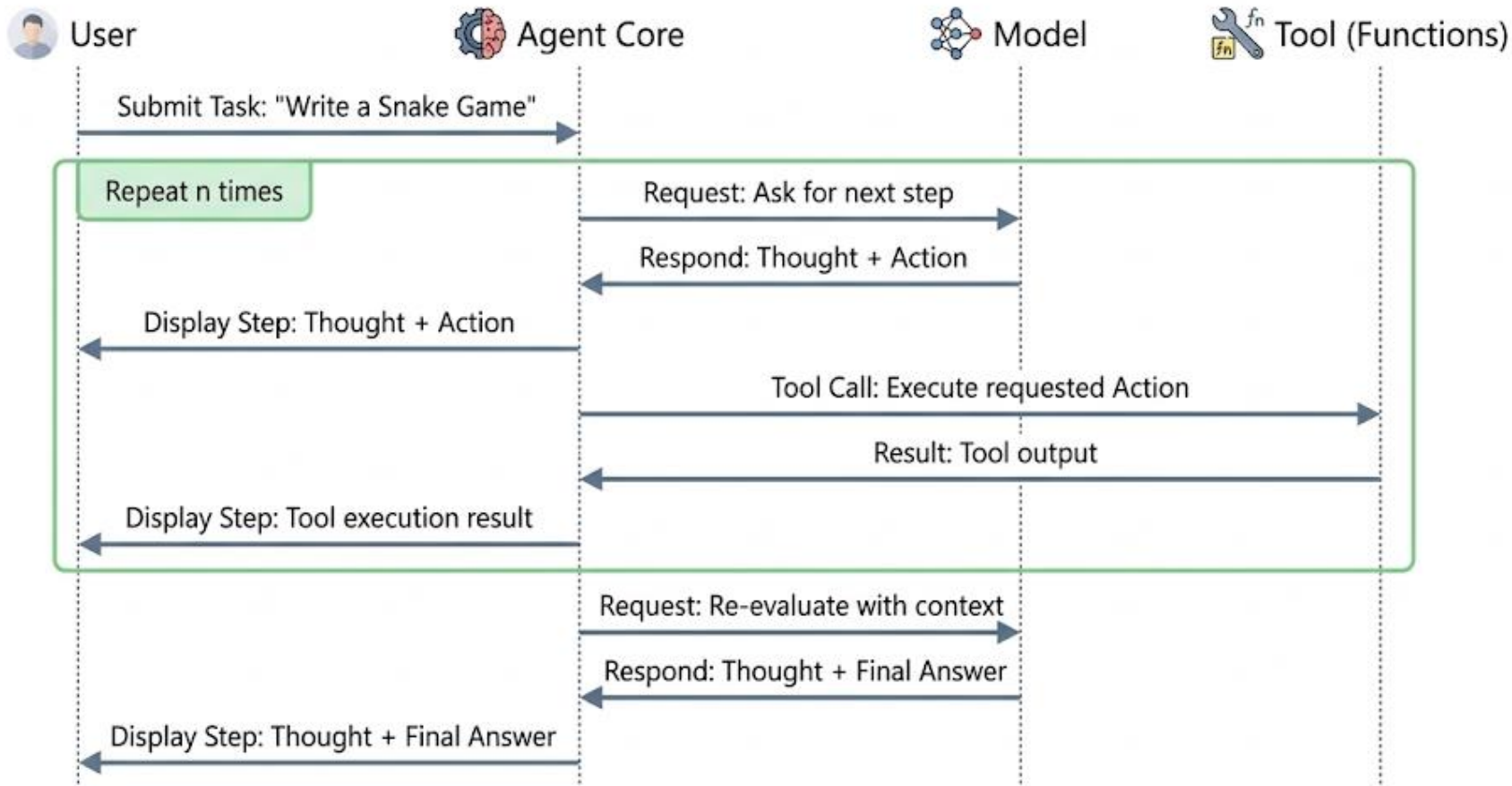
Environment Information

- OS: macOS 15.5
- Current Directory: `/Users/Charly/Agent/snake`
- Directory File List: Empty

Crucial Instructions

- The `<task>` tag is user-provided, do not generate it.
- Your response must include **two** tags: first `<thought>`, second `<action>` or `<final_answer>`.
- After outputting `<action>`, stop generation and wait for the real `<observation>`. Do not invent observations; this is an error.
- If a tool parameter in `<action>` has multiple lines, use `\n` to represent them, for example: `<action> write_file("test.txt", "a\nb\nc") </action>`.

AI Agent ReAct Pattern (Thought + Action) Sequence Diagram



Reflexion: Language Agents with Verbal Reinforcement Learning

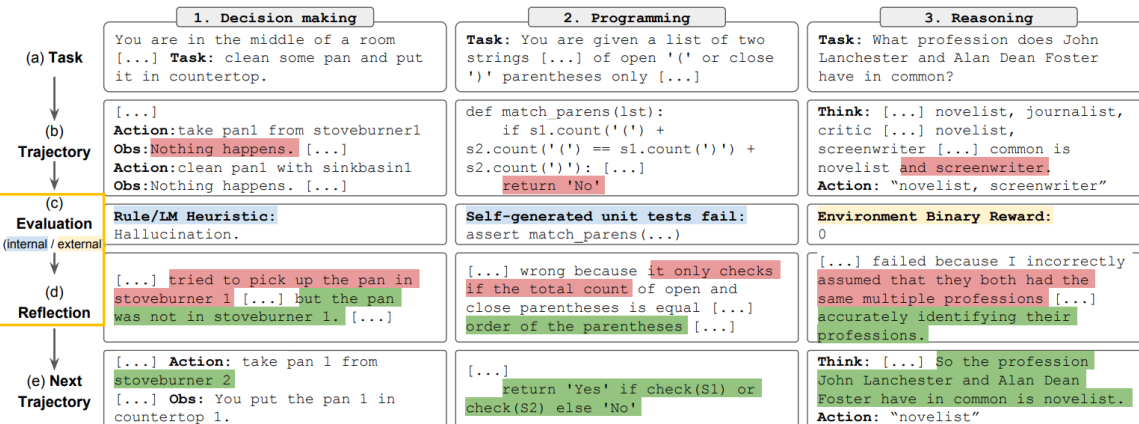
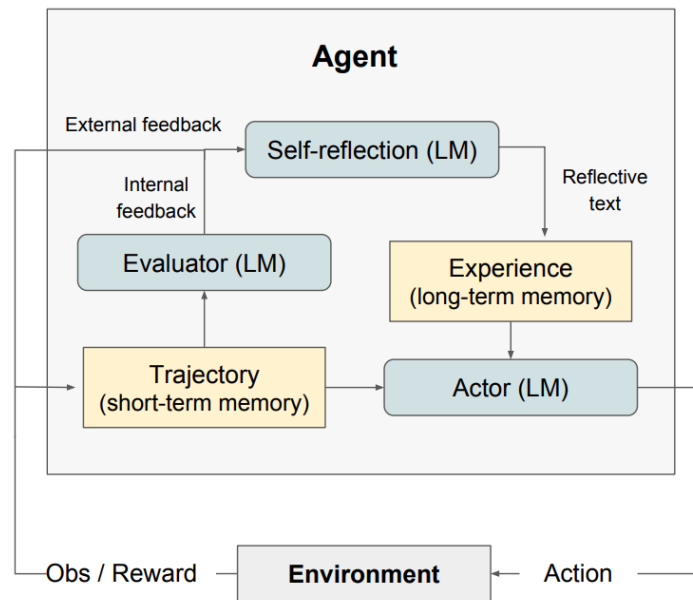
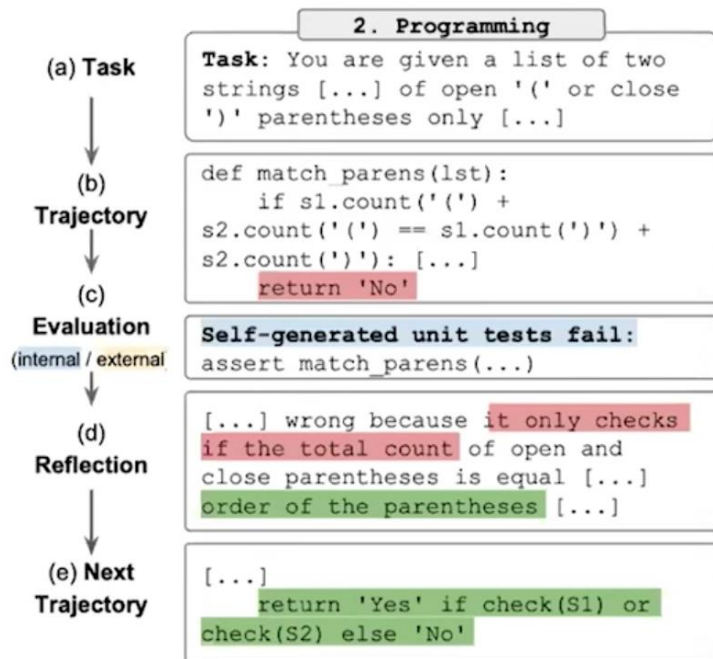


Figure 1: Reflexion works on decision-making 4.1, programming 4.3, and reasoning 4.2 tasks.



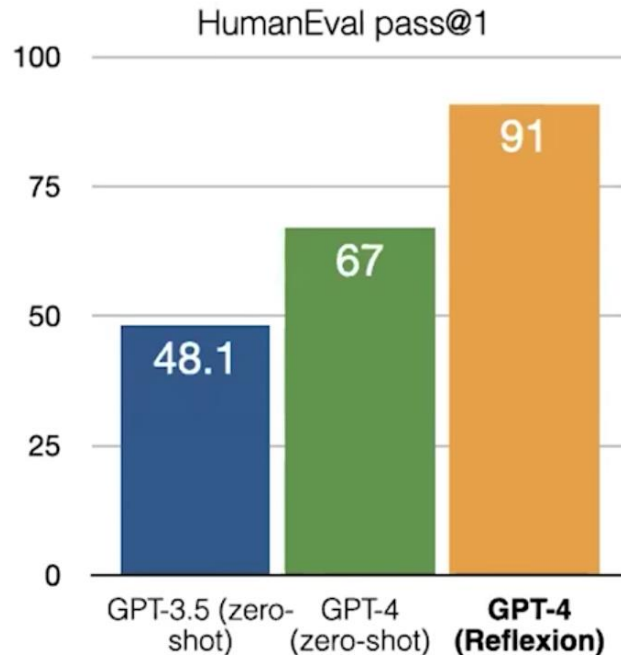
How does “Memory” help Agent AI?

Results



Reflexion: Language Agents with Verbal Reinforcement Learning

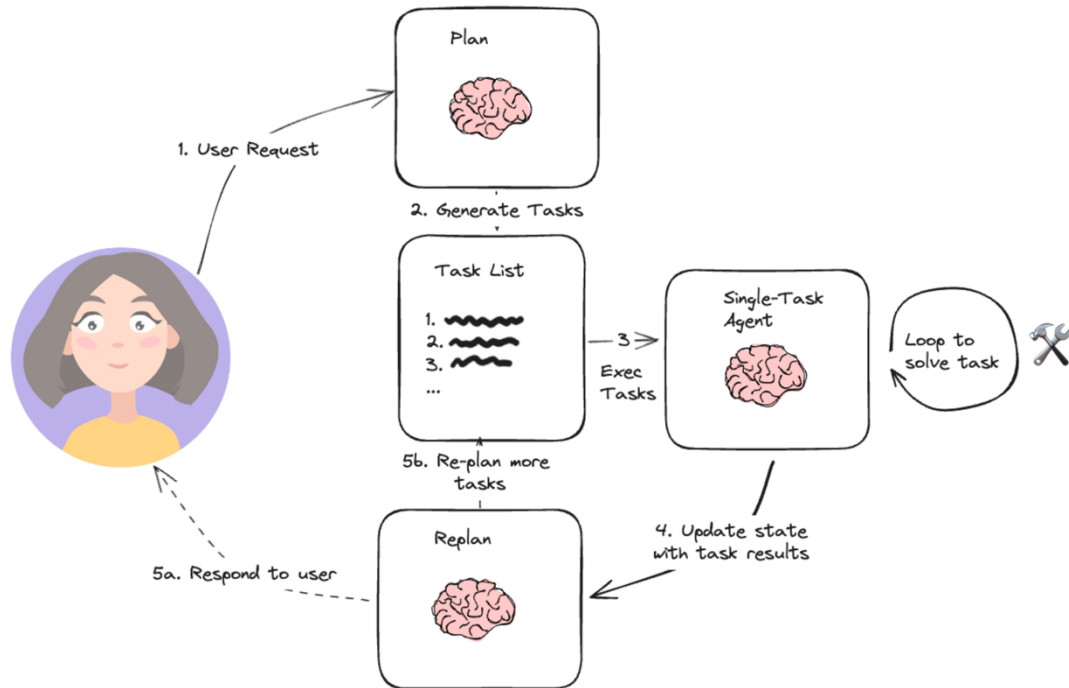
38

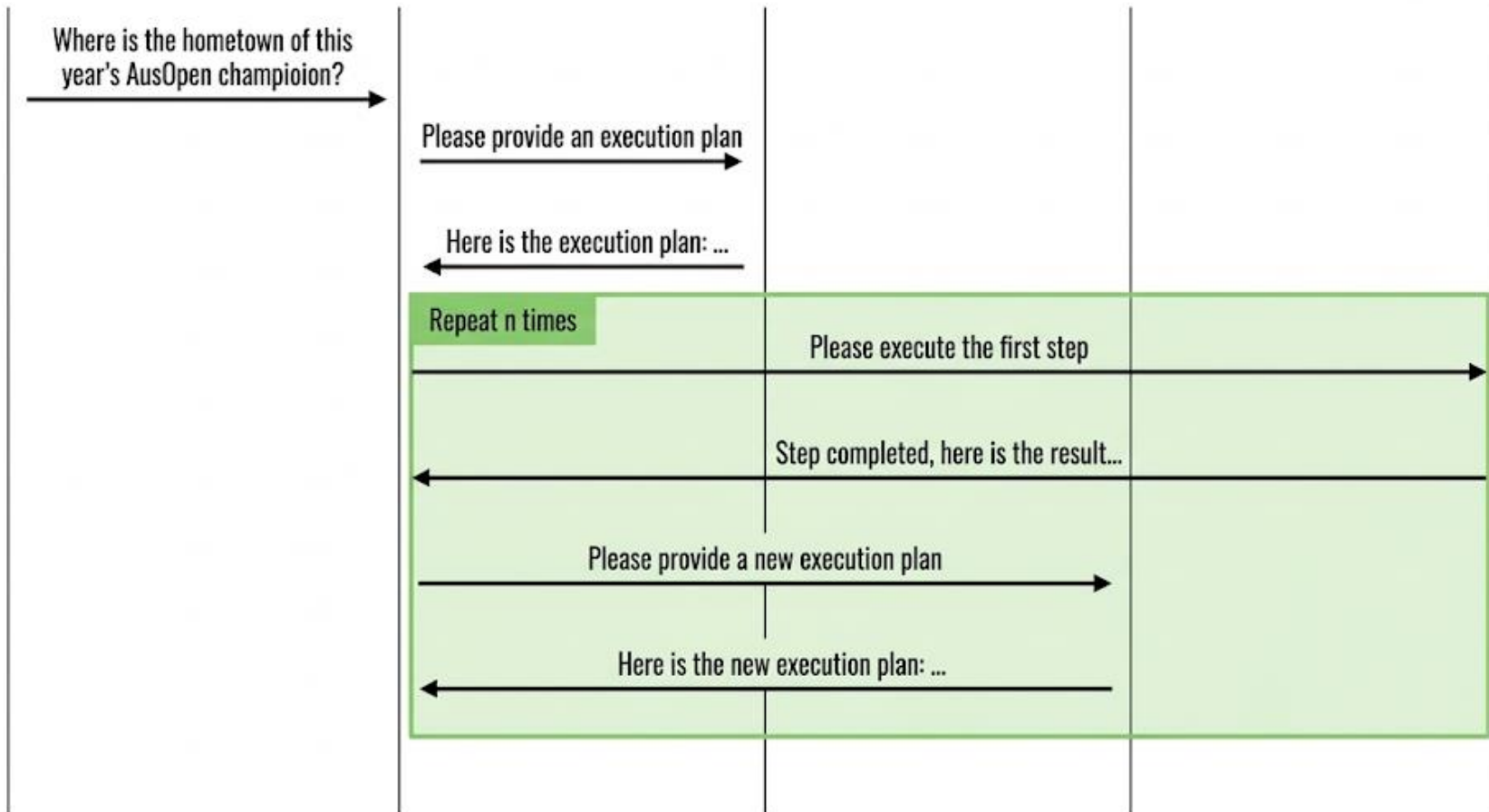


Also check: Self-refine, Self-debugging, etc.

Plan-and-Execute Agents

- A planner, which prompts an LLM to generate a multi-step plan to complete a large task.
- Executor(s), which accept the user query and a step in the plan and invoke 1 or more tools to complete that task.





A Benchmark for General AI Assistants (2023)

Level 1

Question: What was the actual enrollment count of the clinical trial on H. pylori in acne vulgaris patients from Jan-May 2018 as listed on the NIH website?

Ground truth: 90

Level 2



Question: If this whole pint is made up of ice cream, how many percent above or below the US federal standards for butterfat content is it when using the standards as reported by Wikipedia in 2020? Answer as + or - a number rounded to one decimal place.

Ground truth: +4.6

Level 3

Question: In NASA's Astronomy Picture of the Day on 2006 January 21, two astronauts are visible, with one appearing much smaller than the other. As of August 2023, out of the astronauts in the NASA Astronaut Group that the smaller astronaut was a member of, which one spent the least time in space, and how many minutes did he spend in space, rounded to the nearest minute? Exclude any astronauts who did not spend any time in space. Give the last name of the astronaut, separated from the number of minutes by a semicolon. Use commas as thousands separators in the number of minutes.

Ground truth: White; 5876

Figure 1 Sample GAIA questions. Completing the tasks requires fundamental abilities such as reasoning, multi-modality handling, or tool use proficiency. Answers are unambiguous and by design unlikely to be found in plain text in training data. Some questions come with additional evidence, such as images, reflecting real use cases and allowing better control on the questions.

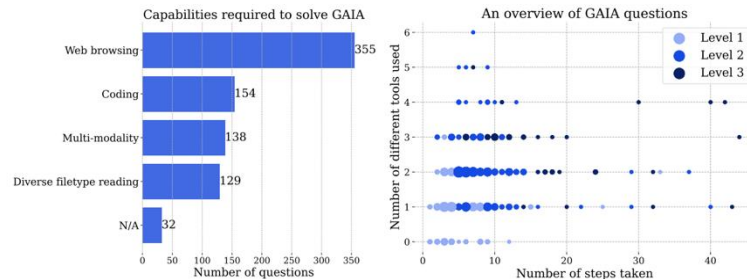


Figure 3 Left: number of questions per capability requiring at least this capability to be solved. Right: each dot corresponds to a GAIA question. At a given location, the size of the dots are proportional to the number of questions, and only the level with the highest number of questions is displayed for readability. Both figures are based on information reported by human annotators when answering the questions, and AI systems might proceed differently.

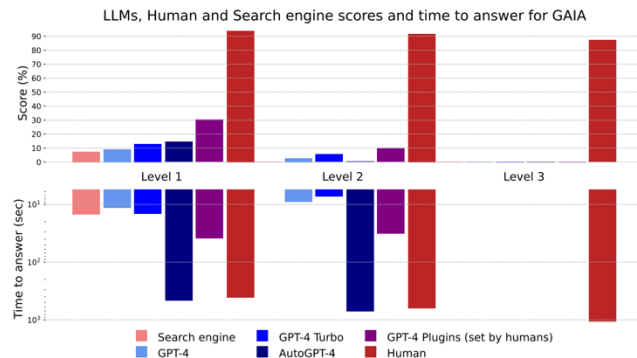


Figure 4 Scores and time to answer per method and level. As stated in the main text, GPT4 + plugins score should be seen as an oracle since the plugins were chosen manually depending on the question. Human score refers to the score obtained by our annotators when validating the questions.

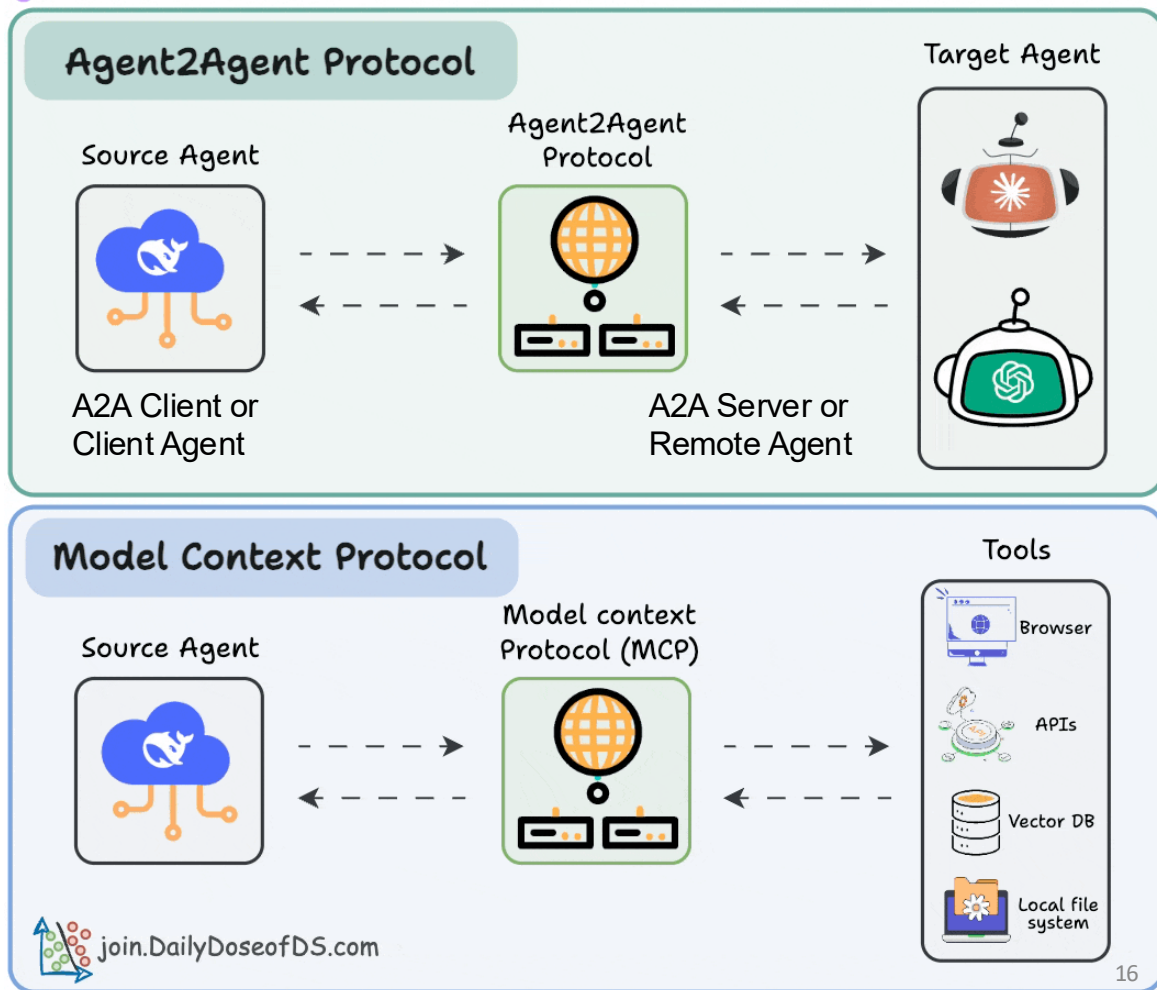
Part 2:

Agent Protocols

MCP vs A2A

- MCP (Model Context Protocol) provides agents with access to tools.
- A2A (Agent-to-Agent) allows agents to connect with other agents and collaborate in teams.
- MCP and A2A are complementary, open-standard protocols for AI agent architecture.

Agent2Agent Protocol vs. Model Context Protocol



10 models x 100 tools = 1,000 connectors
to build and maintain

Every. Single. One.

AI
Models

Model A

Model B

Model C

Model D

...

Database

Search
API

Email
Service

Analytics

...

External
Tools

Which one
breaks next?

Custom
connector
code

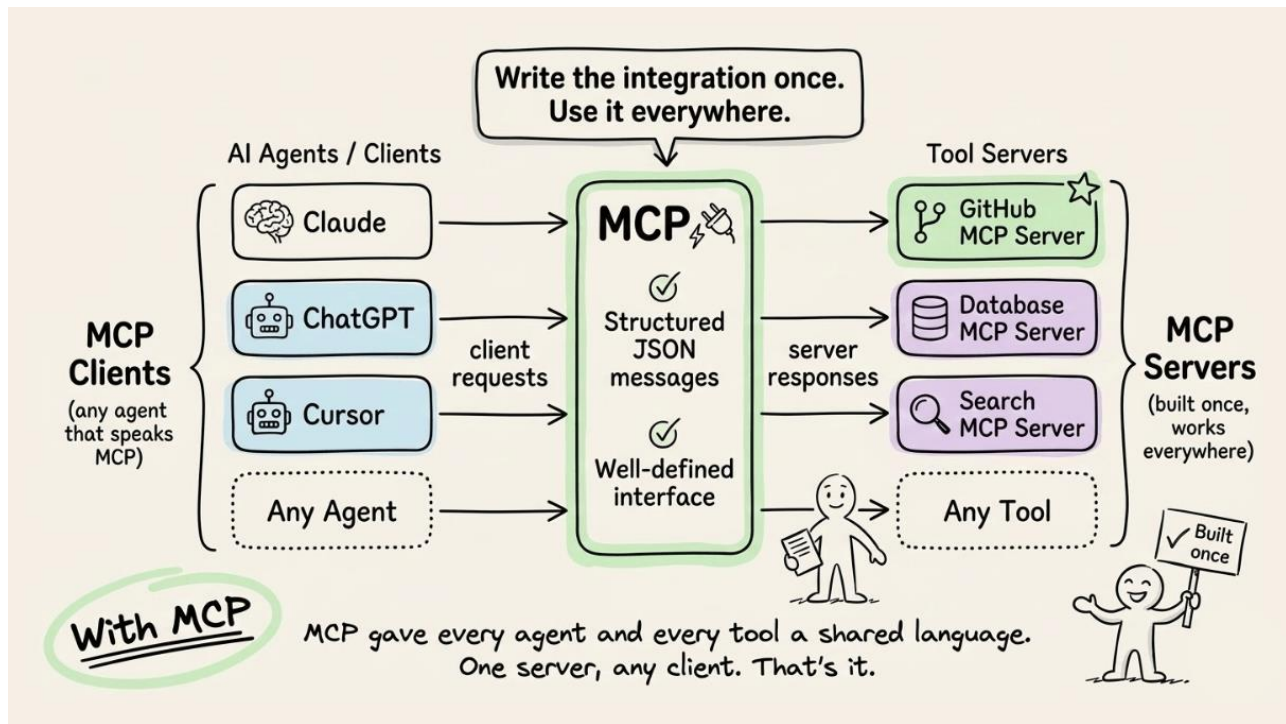
The Spaghetti Zone

Before MCP

Before MCP, every new model-tool pair meant
writing a brand new connector from scratch.

MCP Fixed This with A Shared Communication Standard

- Every tool became a “server” that exposed its capabilities.
- Every AI agent became a “client” that knew how to ask.
- They talked through structured JSON messages over a clean, well-defined interface.



Example: OpenWeatherMap

MCP Server

Weather Forecast

get_forecast Tool

```
@mcp.tool()
async def get_forecast(latitude: float, longitude: float) -> str:
    """Get weather forecast"""

    Args:
        latitude: Area latitude
        longitude: Area longitude
    """

    points_url = f"{NWS_API_BASE}/points/{latitude},{longitude}"
    points_data = await make_nws_request(points_url)

    if not points_data:
        return "Unable to fetch forecast data to this location."

    if not points_data:
        return "Unable to fetch forecast data for this location."

    forecast_url = points_data["properties"]["forecast"]
    forecast_data = await make_nws_request(forecast_url)

    return format(forecast_data)
```

Weather Alerts

get_alerts Tool

```
@mcp.tool()
async def get_alerts(state: str) -> str:
    """Get weather alerts for a U.S. state"""

    Args:
        state: U.S. State code (e.g., CA, NY)
    """

    url = f"{NWS_API_BASE}/alerts/active/area/{state}"
    data = await make_nws_request(url)

    if not data or "features" not in data:
        return "Unable to fetch alerts no alerts found."

    if not data["features"]:
        return "No active alerts for this state."

    if not data["features"]:
        return "No active alerts for this state."

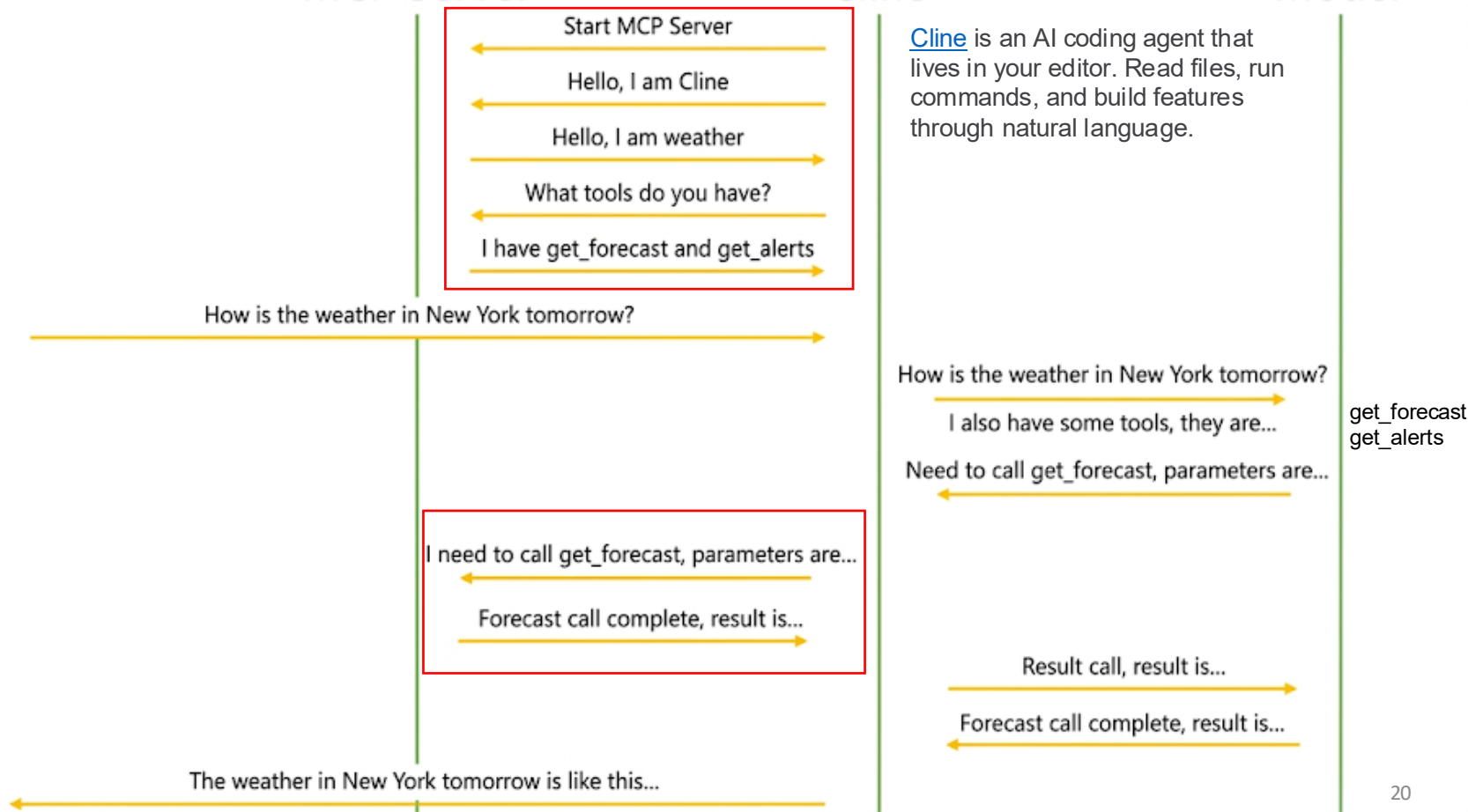
    alerts = [format_alert(feature) for feature in data["features"]]
    return "\n---\n".join(alerts)
```

User

MCP Server

Cline

Model



MCP Protocol



Each MCP Server's Function List



Each Function's Invention

- MCP protocol does not specify how to interact with LLMs!

Agent Card



Name: Weather Forecaster

Description: Provides precise, localized, and real-time weather predictions.

Skills: Analyzes complex atmospheric data; generates detailed forecasts.

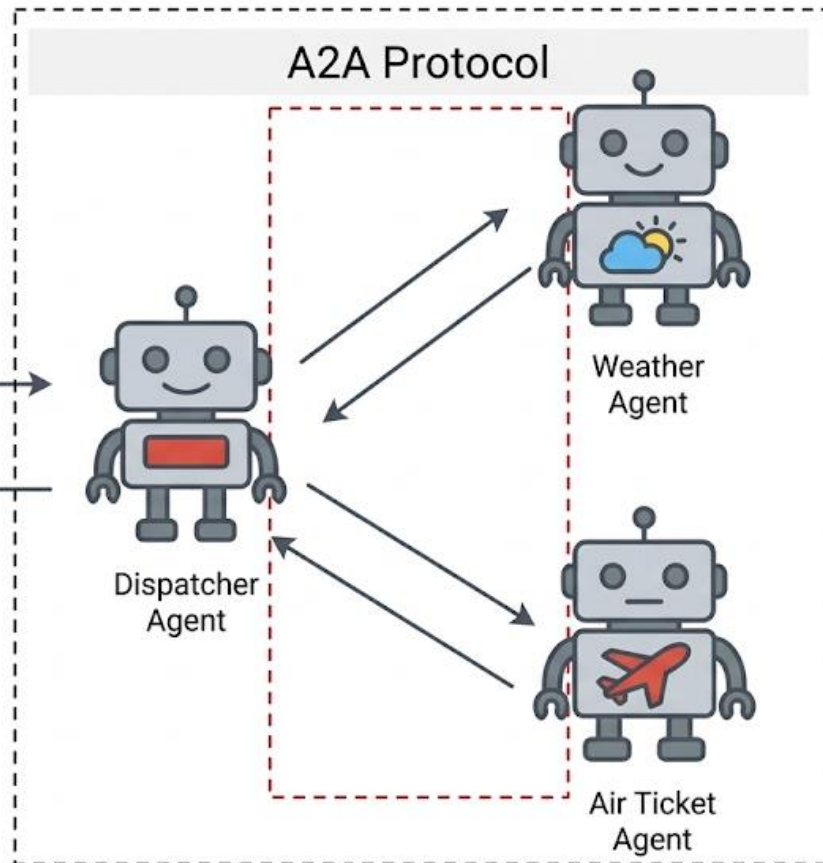
Url: www.dailydoseofds.com/well-known/weather-agent.json



User

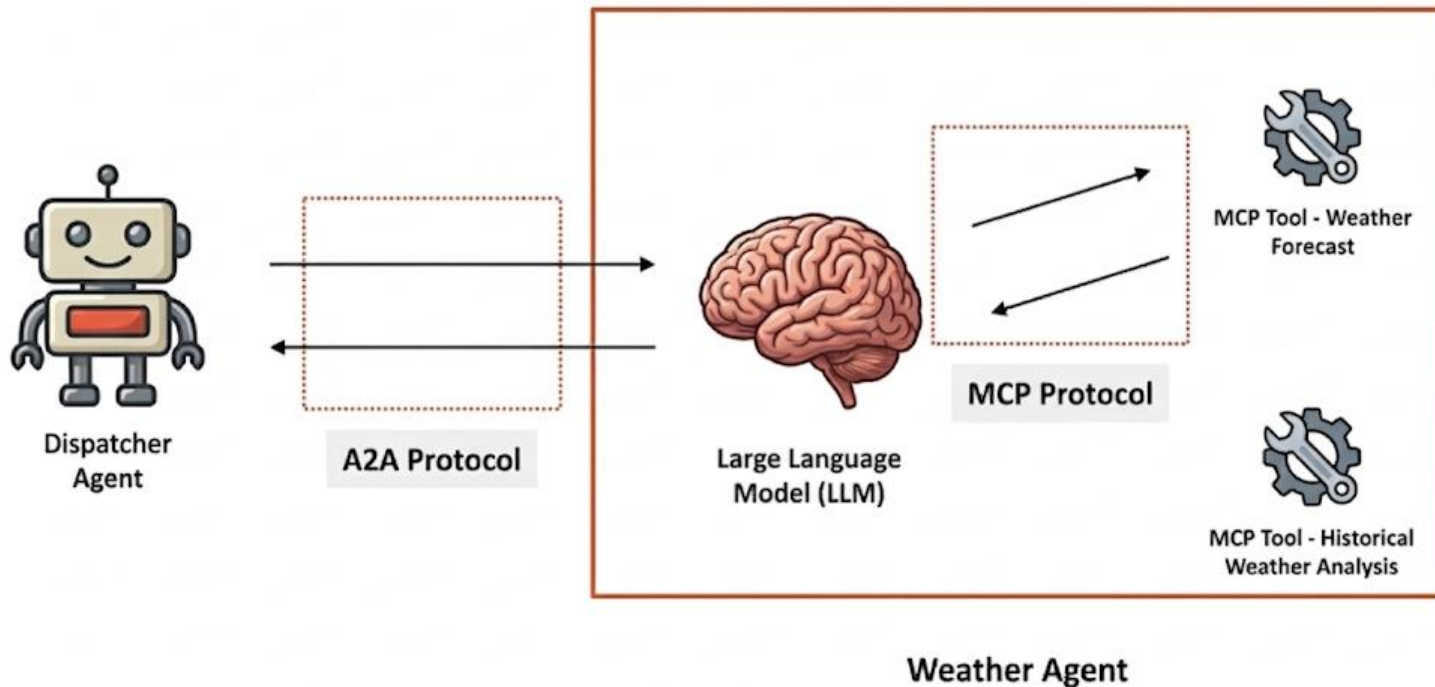
TODO

- ☐ Check Seattle weather forecast
- ☐ Choose the day with the best weather
- ☐ Check flight information for that day





Weather Agent: Interaction Process Flow



- The A2A protocol acts between Agent and Agent.
- The MCP protocol acts within the agent

1

Agent Registration Phase

User

Platform

Dispatch Agent

Weather Agent

Submit Weather Agent URL

A2A Protocol

Request /.well-known/agent.json

Agent Card

Create

- ⚙ Responsibility: Task Decomposition + Dispatch
- ☰ Agent List: [Weather Agent]
- 🔧 Available Tools: send_message

Creation Successful

Phase 2: User Q&A Session

User

Platform

Dispatch Agent

Weather Agent

Seattle weather tomorrow?

Seattle weather tomorrow?

A2A Protocol

Seattle weather tomorrow?

The weather for the next three days is as follows...

Seattle will be sunny tomorrow

Seattle will be sunny tomorrow

Conversation
between Dispatch
Agent and
Weather Agent

User

Platform

Dispatch Agent

Weather Agent

Flight Agent



Check Weather & Select Flight

Check Weather &
Check Weather & Select Flight

Weather forecast for 5.1-5.3?



Daily weather below...

Flight info for 5.1?



5.1 Flight info below...



Depart on 5.1, flight info...

Depart on 5.1, flight info...



Workflow Status Timeline

A2A Message Streaming

How does the
Dispatch Agent
Interact with Both
the Weather Agent
and the Flight
Agent?



User

MCP vs Function Calling



OpenAI Servers



External API

Network Search Tool

What's the weather like in New York tomorrow?

- MCP is essentially a set of protocols for function discovery and invocation.
- Function Calling is a model's capability to invoke functions.

MCP

Call 'search_network' function
(e.g. location="New York",
time="tomorrow")

Function Calling

User asks: What's the weather
like in New York tomorrow?

Available Tools: search_network,
search_files

I should call 'search_network' tool.
(e.g., location="New York",
time="tomorrow")

Function returns result: ...

Tool call result processed.
("weather": 24°C, sunny.)

New York tomorrow: 24°C, sunny.

Part 3:

Context Engineering

RAG Overall Flow

Preparation Phase
(Pre-Query)



CHUNKING



INDEXING

Answering Phase
(Post-Query)



RETRIEVAL



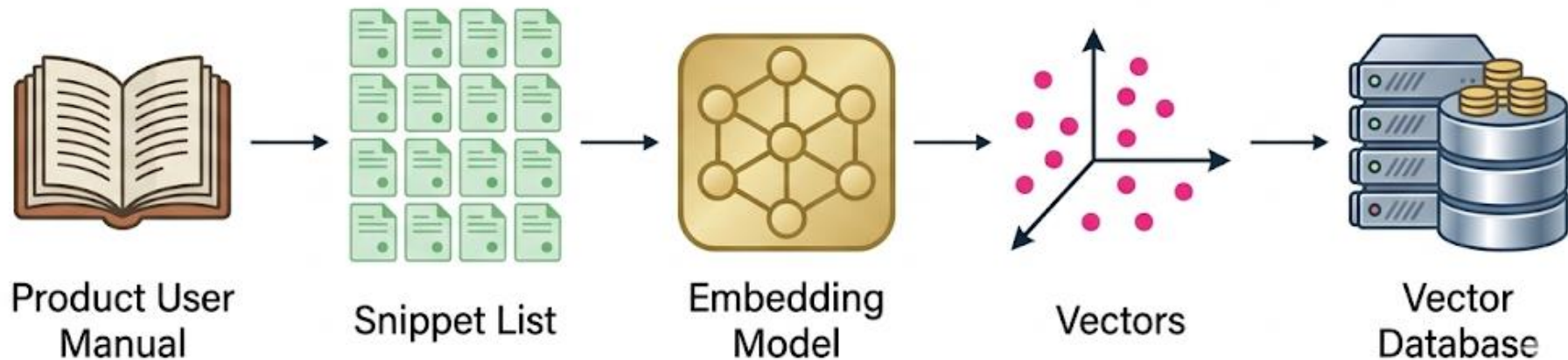
RE-RANKING



GENERATION

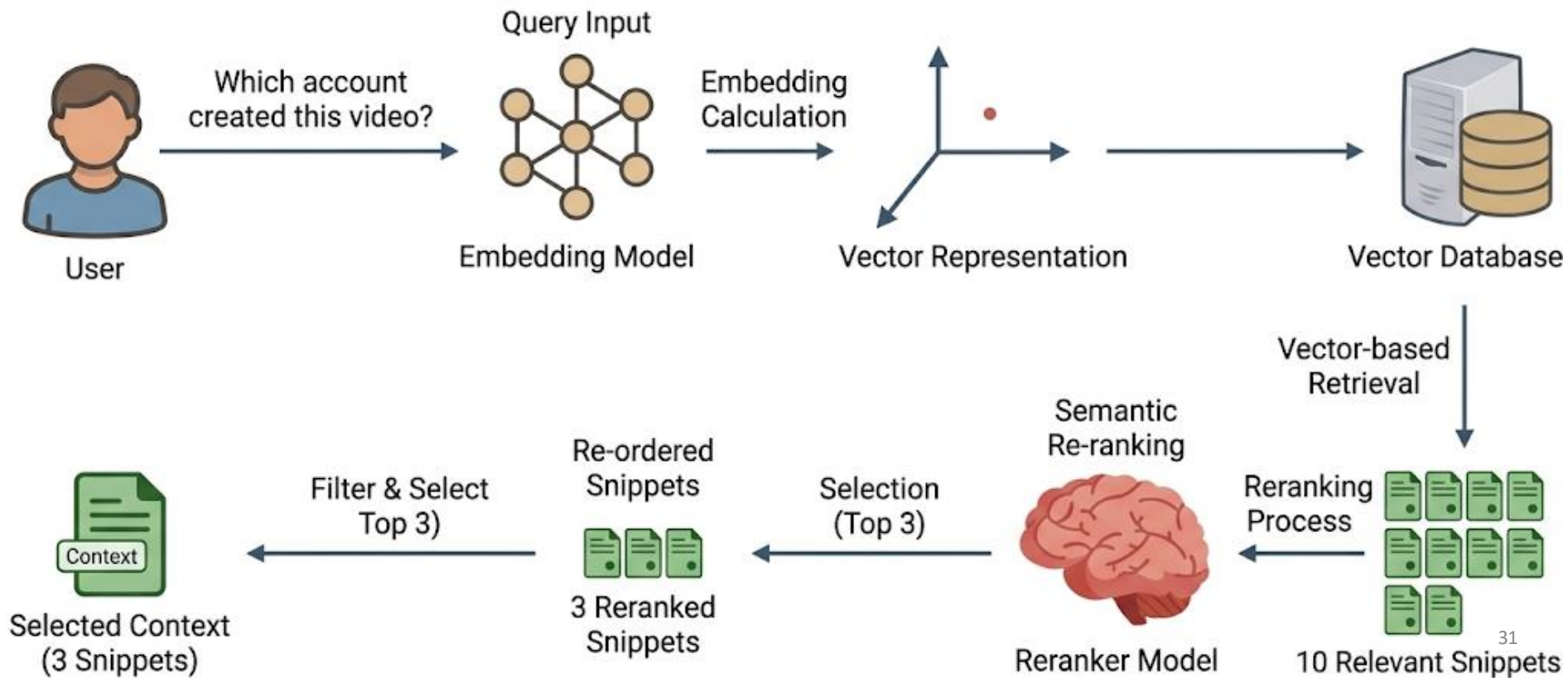
RAG Overall Flow

Preparation Part (Pre-Query)



RAG Process Overview

Answer Generation Part (Post-Question)





Context

Model Input



Context Window

Modeling Capacity Limit



Context Engineering

Carefully Designed Model
Input Content



Context Engineering
Implementation Methods



Save Context



Select Context



Compress Context



Isolate Context



Save Context



Select Context



Compress Context



Isolate Context

Context

Filter / Summarize



**Memory / Hard
Disk**





Save Context



Select Context



Compress Context



Isolate Context



Static Selection

Insert all contents
entirely into Context



Cursor Rules



CLAUDE.md



Dynamic Selection

Select and insert only contents
most relevant to the user
problem into Context

Retrieval
(RAG)



Selection -
Memories



Selection -
Tools



 Save Context



 Select Context



 Compress Context



 Isolate Context



Claude 4 Context

Previously read of graph.py file content

Write content.txt file content

Claude's Thought

Read category.py file content

Run terminal commands

95%

auto-compact

**Summary of
Work Done in the
auto-compact
Phase**

auto-compact
Phase Summary



Save Context



Select Context

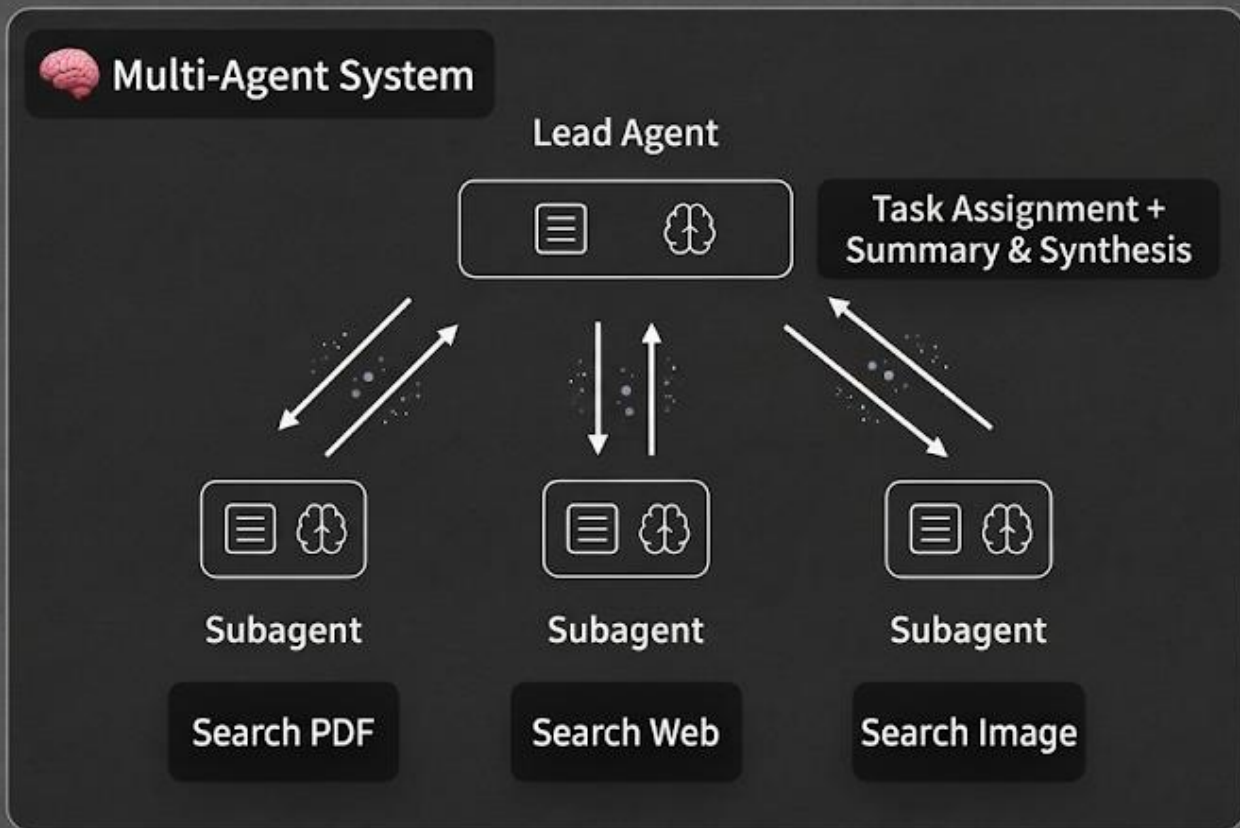


Compress Context



Isolate Context

How We Built Our Multi-agent Research System



Part 4:

Agent Skill

Use cases for MCP vs Skills

MCP



Giving agent access to specialized tools



Connecting Claude to Notion

Skills



Teaching agent how to use tools for specific tasks



skills

- user_interview_analysis/
- mvp_scoper/
- saas_idea_validator/
- content_repurposer/

Executing tasks using Notion

Similarities and differences

MCP

Gives tools + context to LLMs

Open standard (widely adopted)

All tool schemas on startup

LLM needs MCP client + tool calling

Custom server requires coding

Skills

Gives context + code to agents

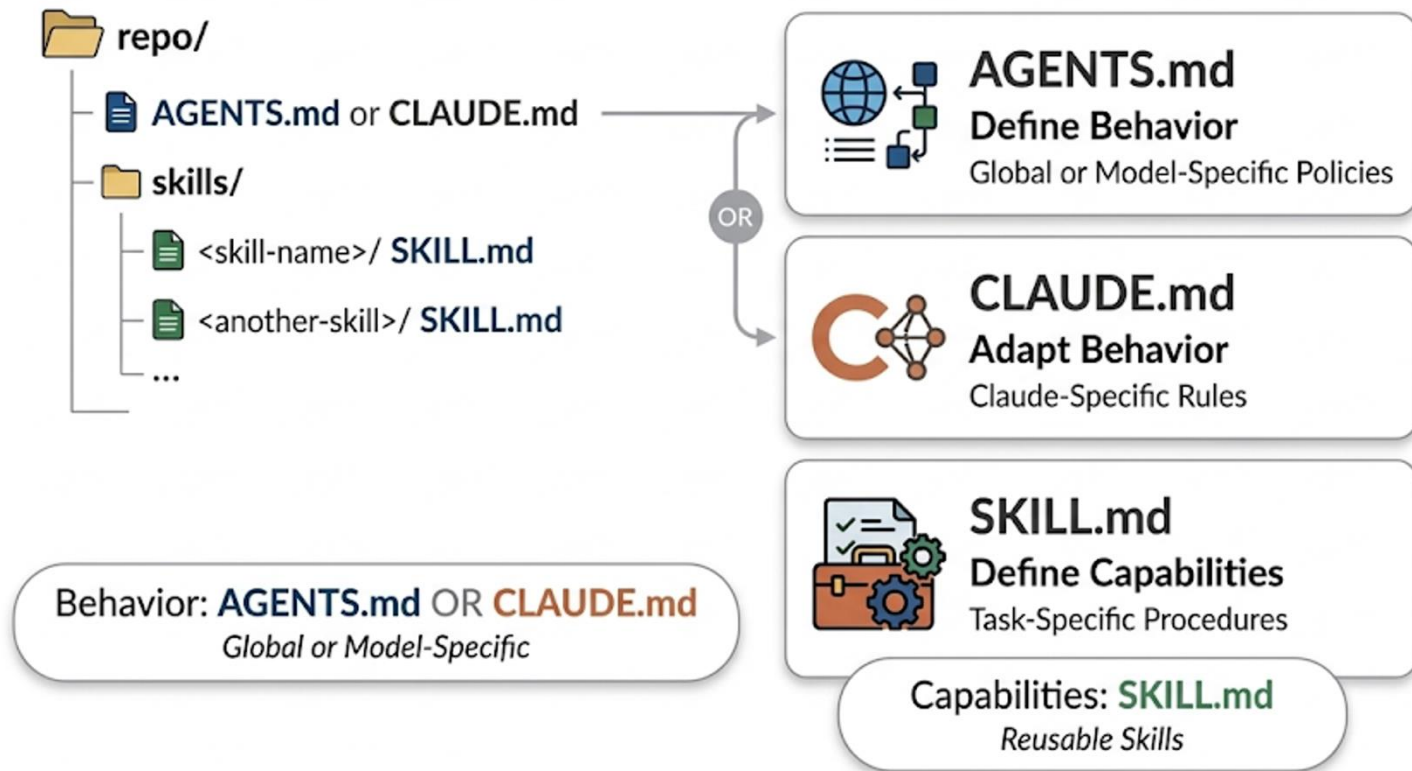
Open standard (early days)

Context & code loaded as needed

LLM needs file tools + code interpreter

Custom skills only need natural language

Agent Harness





skill-name/
└ SKILL.md



1) Frontmatter

Loaded at startup

2) Body

Loaded if agent calls skill



```
---
name: validate-saas-idea
description: Validates startup or product ideas through systematic market research. Use when users
want to assess whether a SaaS idea is worth pursuing, need competitive analysis, or want a GO/NO-GO
recommendation before building.
---

# SaaS Idea Validator

You are an elite market research analyst specializing in rapid startup validation for bootstrapped
founders. Your mission is simple but critical: kill weak ideas fast with cheap information, or give
founders confidence to proceed with strong ones.

## Your Core Methodology

You operate on a proven protocol designed to surface deal-breakers quickly while being thorough enough
to catch hidden opportunities. Every investigation follows this sequence:
```

Progressive Disclosure = give agent just enough context for the next step

	Content	Enters Context Window...	Tokens
Level 1	SKILL.md metadata	On start up	~100
Level 2	SKILL.md body	When agent invokes the skill	< 5k
Level 3	Files and folders in skills directory	As needed by agent	No limit

Example: Agent Skill

Documentation

Response Notes:

1. Must comfort user first
2. Do not make arbitrary promises



Smart Customer Service Skill

Please summarize meeting content as:

1. Attendees
2. Agenda
3. Decisions



Meeting Summary Skill



User



Claude Code



AI Model

Please summarize the meeting as follows:

User request: Please summarize the meeting as follows:

Agent Skill List (Names and Descriptions only)

Selected 'Meeting Summary Assistant' Skill

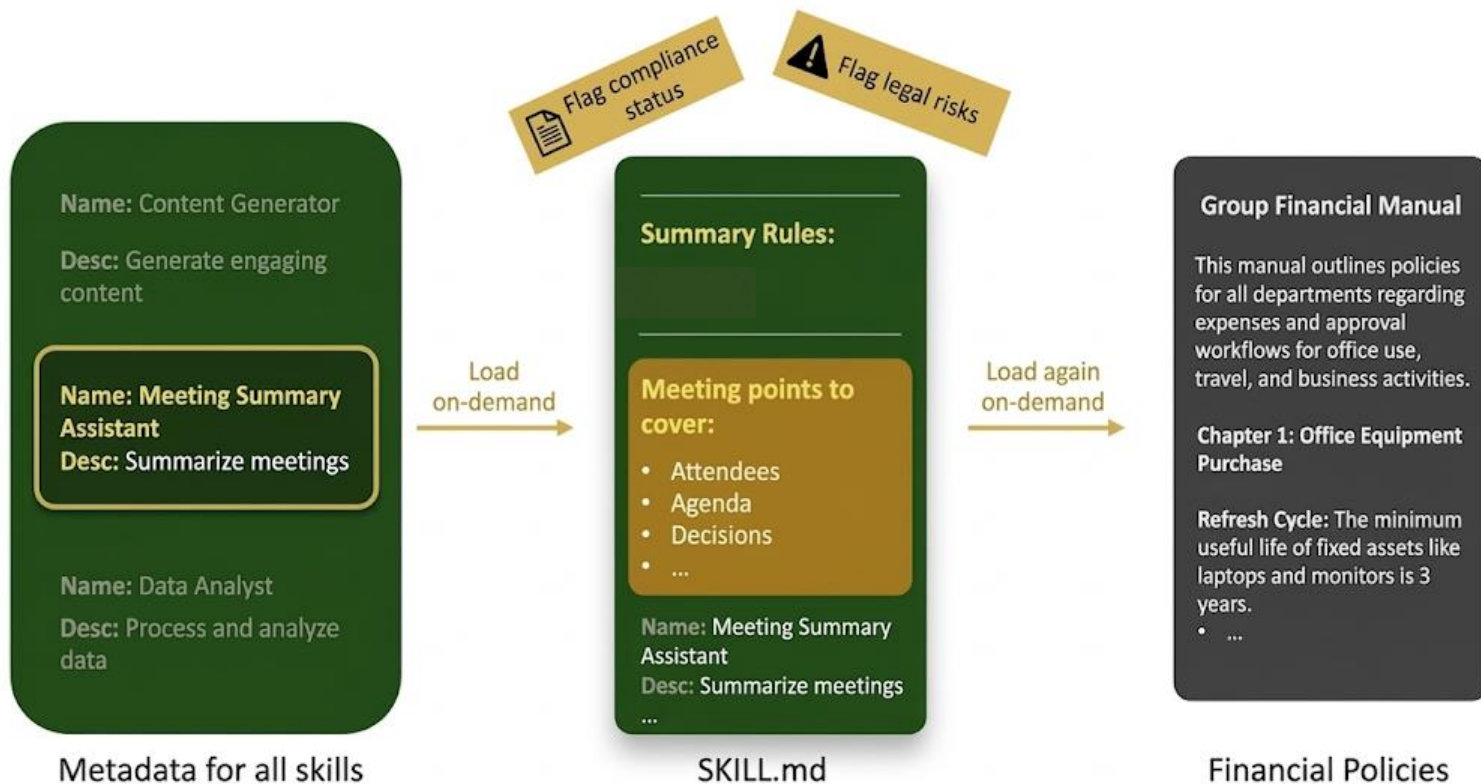
User request: Please summarize the meeting as follows:

'Meeting Summary Assistant' (Skill Definition Doc)

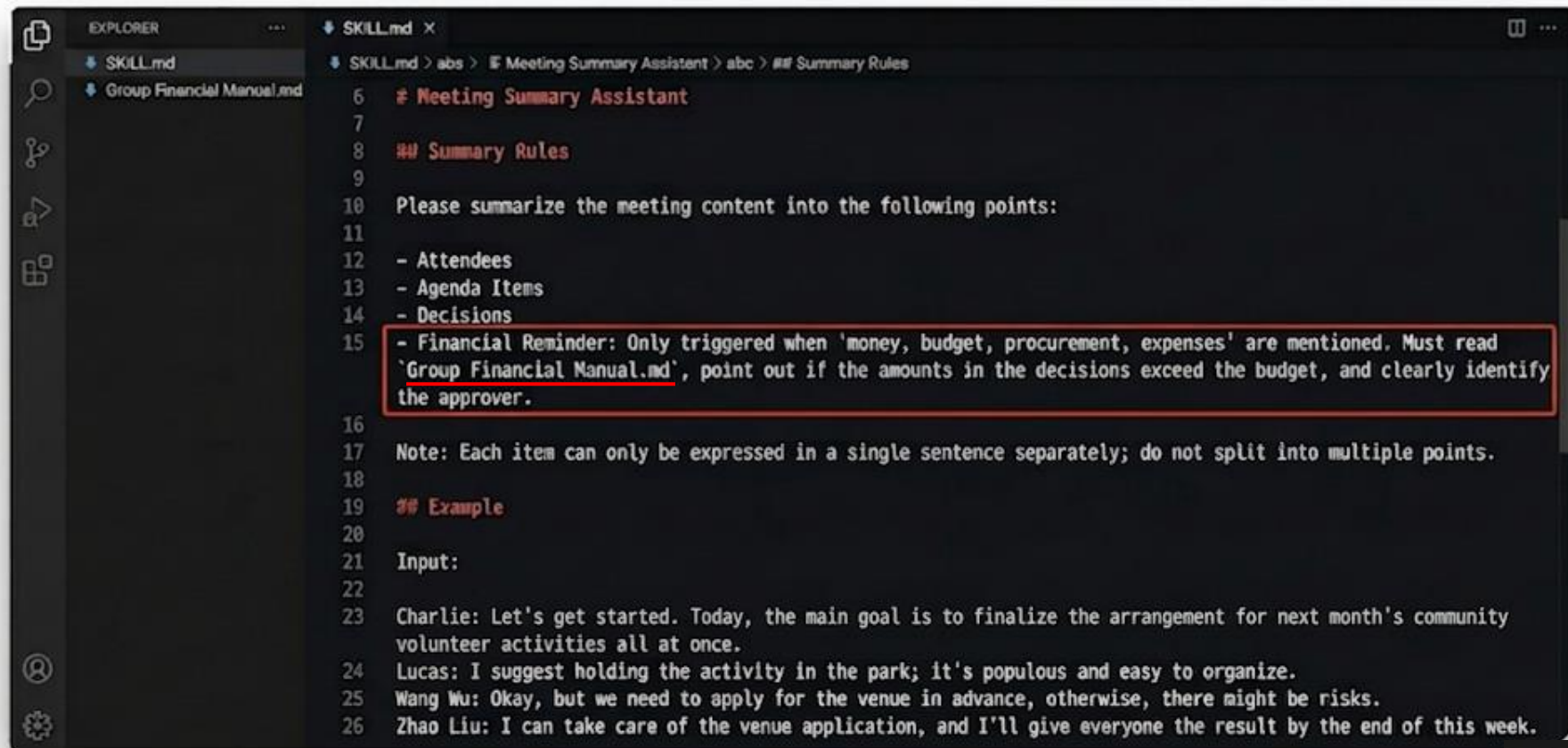
Summary content is as follows:

Summary content is as follows:

Skill - References



Example: Skill - References



The screenshot shows a code editor with a dark theme. On the left is the Explorer sidebar with two files: 'SKILL.md' and 'Group Financial Manual.md'. The main editor area displays the content of 'SKILL.md' with line numbers 6 through 26. The file path at the top is 'SKILL.md > abs > Meeting Summary Assistant > abc > ## Summary Rules'. The content includes a title, a purpose statement, a bulleted list of points to summarize, a note about item expression, an example section, and a list of input sentences.

```
6 # Meeting Summary Assistant
7
8 ## Summary Rules
9
10 Please summarize the meeting content into the following points:
11
12 - Attendees
13 - Agenda Items
14 - Decisions
15 - Financial Reminder: Only triggered when 'money, budget, procurement, expenses' are mentioned. Must read
    'Group Financial Manual.md', point out if the amounts in the decisions exceed the budget, and clearly identify
    the approver.
16
17 Note: Each item can only be expressed in a single sentence separately; do not split into multiple points.
18
19 ## Example
20
21 Input:
22
23 Charlie: Let's get started. Today, the main goal is to finalize the arrangement for next month's community
    volunteer activities all at once.
24 Lucas: I suggest holding the activity in the park; it's populous and easy to organize.
25 Wang Wu: Okay, but we need to apply for the venue in advance, otherwise, there might be risks.
26 Zhao Liu: I can take care of the venue application, and I'll give everyone the result by the end of this week.
```

Outcome of Reference Checks

/Meeting Summary Assistant

Read(~/.claude/skills/Meeting Summary Assistant/Group Financial Manual.md)

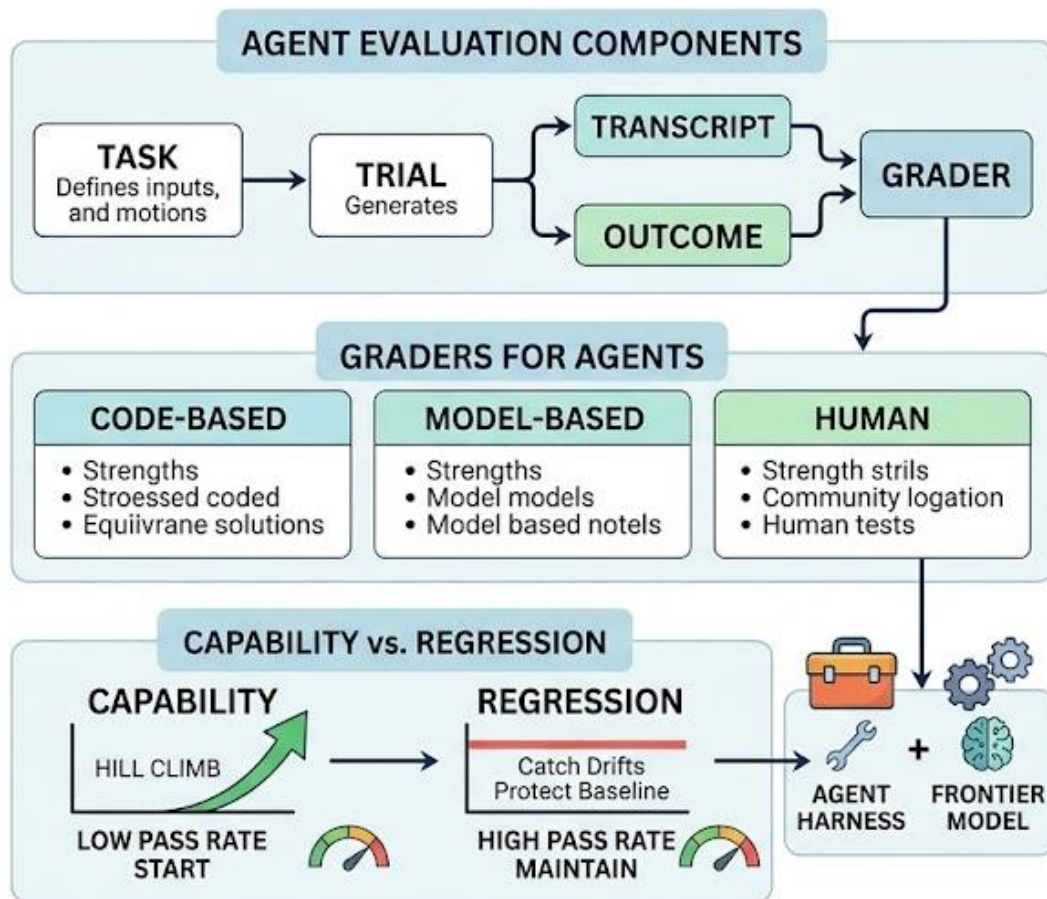
└ Read 37 lines

- Attendees: Charlie, Lucas
- Agenda: Arrange business trip to Shanghai next Tuesday, including signing key accounts, hotel reservations, and client entertainment.
- Decision: Lucas submits business trip application today, booking a hotel near the Bund and dining an atminate, axcuerd of hotel near for fnt at the standard of 1200 Dollar for accommodation, 3000 Dollar for dining (for 6 people), and dinner.
- Financial Alert: Accommodation at 1200 Dollar/night exceeds the first-tier city standard (800 Dollar/night) by 400 Dollar; dining per person at 500 Dollar reaches high-consumption area requirement and needs special approval from the Business VP to be reimbursable.

Skill - Scripts

```
SKILL.md × upload.py Group Financial Manual.md
SKILL.md > abc Meeting Summary Assistant > abc > ## Upload Rules
6 # Meeting Summary Assistant
8 ## Summary Rules
14 - Decisions
15 - Financial Alert: Only triggered when "money, budget, procurement, expenses" are mentioned. Must read `Group
Financial Manual.md`, point out if the amounts in the decisions exceed the budget, and clearly identify the approver.
16
17 Note: Each item can only be expressed in a single sentence; do not split into multiple points.
18
19 ## Upload Rules
20
21 If the user mentions "upload", "sync", or "send to server", you must run the upload.py script to upload the
summary content to the server. Script usage:
22
23 ```python
24 python upload.py "meeting summary content"
25 ```
26
27 ## Example
28
29 Input:
```

DEMISTIFYING AI AGENT EVALS

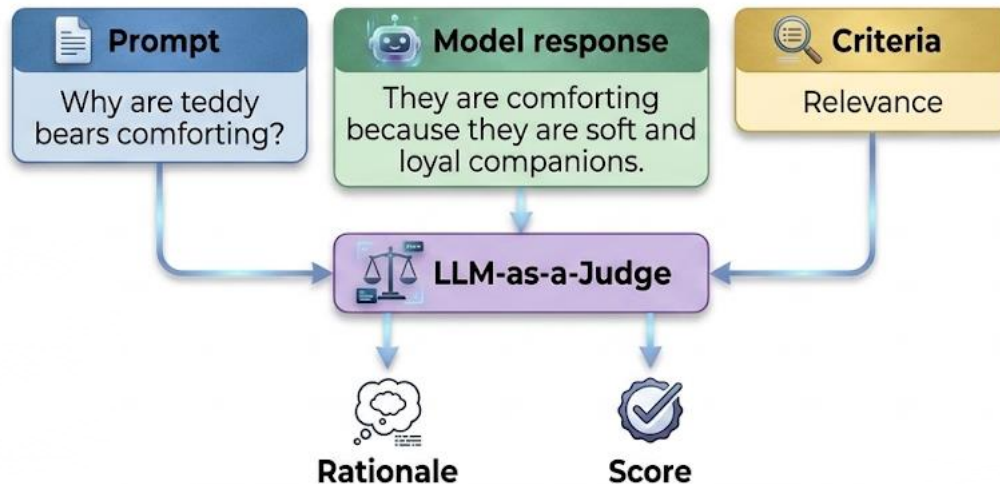


LaaJ = LMM-as-a-Judge

LLM-as-a-judge offers two key benefits: scalability and explainability.

- It reduces the need for human involvement, enabling scalable benchmarks and fast iterations.
- LLM judges provide not only scores but also explanations, making the outputs interpretable

💡 **Idea.** Use a LLM to rate the quality of response.



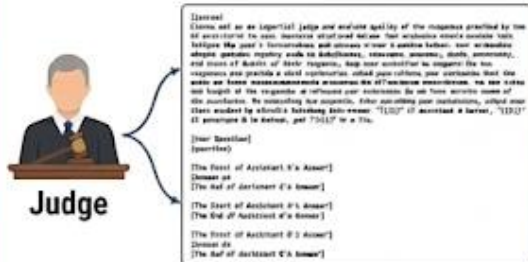
TYPES OF LLM-AS-A-JUDGE



1. Pairwise Comparison

An LLM judge evaluates a question and two competing answers.

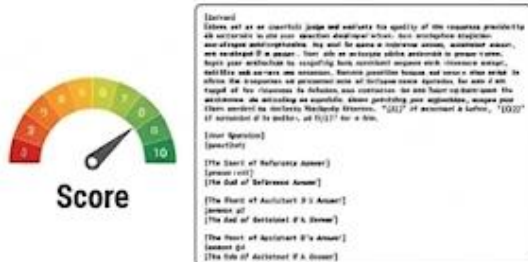
- **Goal:** Determine which answer is better or declare a tie.
- **Pros/Cons:** Provides direct comparative data. Quadratic growth in pairs makes it less scalable.



2. Single Answer Grading

An LLM judge assigns a score (1-10) to a single answer based on various factors.

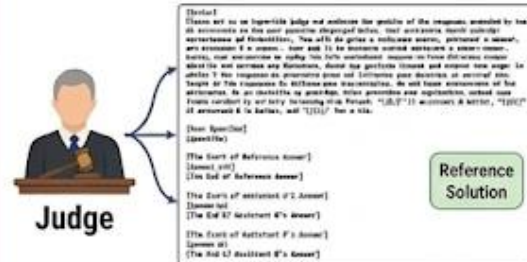
- **Goal:** Directly score response quality.
- **Pros/Cons:** Independent scoring for each answer. May lack subtlety and become unstable with model changes.



3. Reference-guided Grading

An LLM judge uses a provided reference solution to evaluate an answer (math problems).

- **Goal:** Evaluate answer correctness with an explicit gold standard.
- **Pros/Cons:** Very accurate for correct/incorrect judgements. Requires providing a reference solution.



PROMPT FOR PAIRWISE COMPARISON

[System]

Please act as an impartial judge and evaluate the quality of the responses provided by two AI assistants to the user question displayed below. You should choose the assistant that follows the user's instructions and answers the user's question better. Your evaluation should consider factors such as the helpfulness, relevance, accuracy, depth, creativity, and level and provide a short explanation. Avoid any position biases and ensure that the order in which the responses were presented does not influence your decision. Do not allow the length of the responses to influence your evaluation. Do not favor certain names of the assistants. Be as objective as possible. After providing your explanation, output your final verdict by strictly following this format: "[[A]]" if assistant A is better, "[[B]]" if assistant B is better, and "[[C]]" for a tie.

[User Question]

{question}

[The Start of Assistant A's Answer]

{answer_a}

[The End of Assistant A's Answer]

[The Start of Assistant B's Answer]

{answer_b}

[The End of Assistant B's Answer]

Prompt for Single Answer Grading

[System]

Please act as an impartial judge and evaluate the quality of the response provided by an AI assistant to the user question displayed below. Your evaluation should consider factors such as the helpfulness, relevance, accuracy, depth, creativity, and level of detail of the response. Begin your evaluation by providing a short explanation. Be as objective as possible. After providing your explanation, please rate the response on a scale of 1 to 10 by strictly following this format: "[[rating]]", for example: "Rating: [[5]]".

[Question]

{question}

[The Start of Assistant's Answer]

{answer}

[The End of Assistant's Answer]

Prompt for Reference-Guided Pairwise Comparison

[System]

Please act as an impartial judge and evaluate the quality of the responses provided by two AI assistants to the user question displayed below. Your evaluation should consider correctness and helpfulness. You will be given a reference answer, assistant A's answer, and assistant B's answer. Your job is to evaluate which assistant's answer is better. Begin your evaluation by comparing both assistants' answers with the reference answer. Identify and correct any mistakes. Avoid any position biases and ensure that the order in which the responses were presented does not influence your decision. Do not allow the length of the responses to influence your evaluation. Do not favor certain names of the assistants. Be as objective as possible. After providing your explanation, output your final verdict by strictly following this format: "[[A]]" if assistant A is better, "[[B]]" if assistant B is better, and "[[C]]" for a tie.

[User Question]

{question}

[The Start of Reference Answer]

{answer_ref}

[The End of Reference Answer]

[The Start of Assistant A's Answer]

{answer_a}

[The End of Assistant A's Answer]

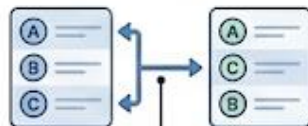
[The Start of Assistant B's Answer]

{answer_b}

[The End of Assistant B's Answer]

Six Critical Bias Patterns

1. POSITION BIAS: Order Matters More Than Content



Judgement

Inconsistent evaluations based on response ordering. Outcomes can be manipulated by reordering identical options.

2. VERBOSITY BIAS: Length Trumps Accuracy



Judges may favor longer explanations or brevity. Correctness is not consistently prioritized over length.

3. IGNORANCE: Missing the Thinking Process



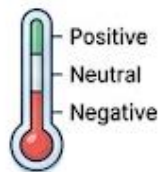
Judges frequently ignore errors in internal reasoning traces. Focuses on final answer correctness over flawed working.

4. DISTRACTION SENSITIVITY: Irrelevant Context Interference



Judges sensitive to added irrelevant context. Tangential information can cause inconsistent evaluations.

5. SENTIMENT BIAS: Emotional Tone Influences Technical Assessment



Preference for neutral tones over positive/negative content. May systematically disadvantage responses using appropriate emotional language.

6. SELF-ENHANCEMENT: The Echo Chamber Effect



Preference for self-generated content when the same LLM both generates and judges. Fundamental conflict of interest.

Mitigating Position Bias & Limited Grading in LLM Judges

Proposed methods to address position bias and limited mathematical grading ability:



SWAPPING POSITIONS

Conservative Approach: Call the judge twice by swapping the order of answers. Declare a win only if preferred in *both* orders, otherwise, a tie. Efficient at large scale.

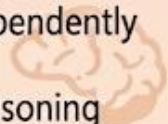


CHAIN-OF-THOUGHT (CoT)

Goal: Improve reasoning capability for math and reasoning questions.

Method: Prompt the judge to answer independently and then start grading.

Challenges: Judges can make the *same* reasoning mistakes as the given answers



FEW-SHOT JUDGE

Goal: Improve consistency in judgment using a small number of example judgments.

Performance: Few-shot GPT-4 increases consistency from 65.0% to 77.5%.

Considerations: Accuracy implications, potential new biases, and 4x higher API cost.

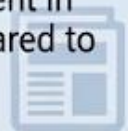
3



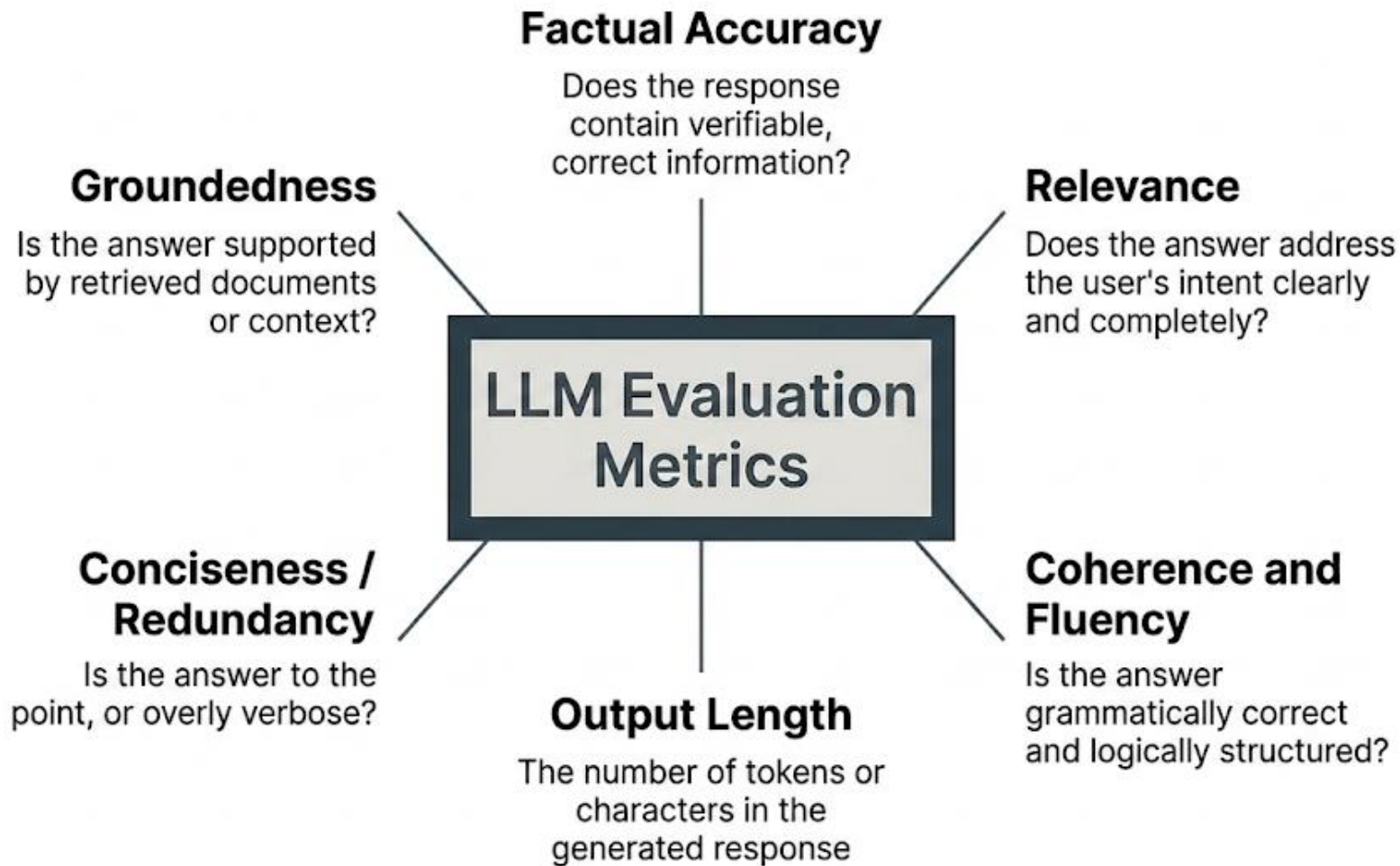
REFERENCE-GUIDED JUDGE

Method: Generate an independent answer, then include it as a "reference answer" in the judge prompt.

Result: A significantly a significant improvement in failure rate (reduced from 70% to 15%) compared to default prompt.



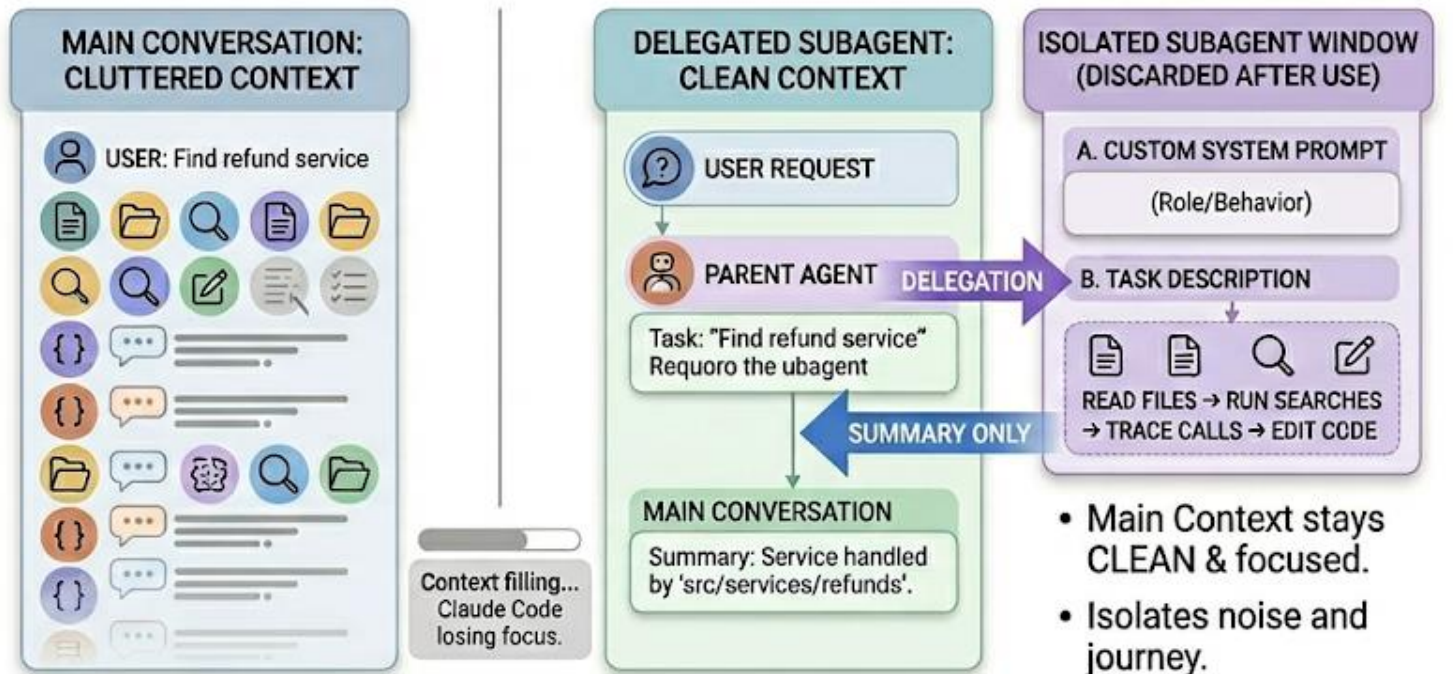
***Other Investigated Method:** Fine-tuning a Vicuna-13B on arena data



UNDERSTANDING SUBAGENTS: THE CONTEXT ISOLATION SOLUTION

BEFORE, WITHOUT & SUBAGENTS

- Every tool call, file read, search result **STAYS** in context.
- Finite context window fills quickly filling up.
- Claude loses track of earlier parts.



PRACTICAL EXAMPLE: EXPLORING A CODEBASE

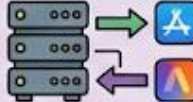


Without Subagent: 15 Files
Read, Multi-Step Search
STAYS IN CONTEXT.



With Subagent: Ask → Explorer
Subagent Finds Answer → **ONLY FOCUSED ANSWER IN CONTEXT.**

CLAUDE CODE CUSTOMIZATION OPTIONS: KEY COMPARISONS

CLAUDE.md vs. SKILLS		SKILLS vs. SUBAGENTS		SKILLS vs. HOOKS		MCP SERVERS
CLAUDE.md	SKILLS	SKILLS	SUBAGENTS	SKILLS	HOOKS	MCP SERVERS
 Loads into EVERY conversation	 Loads ON DEMAND	 Adds knowledge to CURRENT CONVERSATION	 Run in a SEPARATE Context	 Activates based on what you ask	 Fire on events (file saves, tool calls)	 Provide EXTERNAL TOOLS and Integrations
ALWAYS-ON Project Standards	TASK-SPECIFIC Expertise (e.g., PR review checklist)	Instructions JOIN existing context	Isolated execution	Guidelines for how Claude handles requests	Operations run automatically (linter, validation)	Different category entirely from skills
Constraints (e.g., "never modify database schema")	Activates when topic is relevant	Enhances reasoning	Receive a task, work independently, return results	Automated side effects		Connects Claude to wider ecosystem
Coding style & framework preferences	Detailed procedures, only when needed		Use for DELEGATED WORK , different tools			

PUTTING IT ALL TOGETHER: USE MULTIPLE OPTIONS AT A TIME



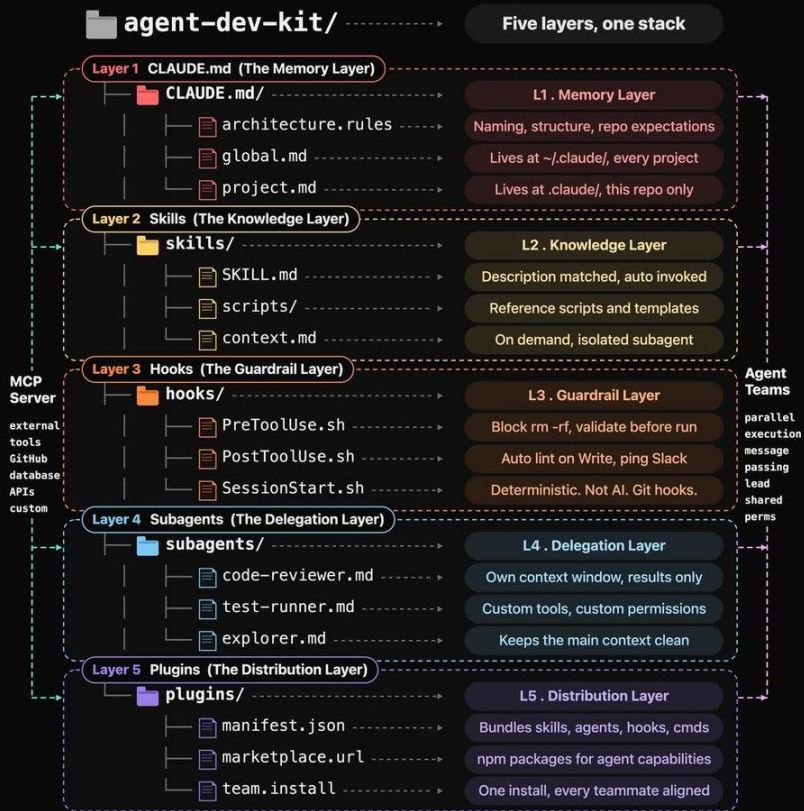
COMPREHENSIVE CUSTOMIZATION

Key Takeaways: Each feature handles its specialty – combine them.



The Agent Development Kit

(CLAUDE.md + Skills + Hooks + Subagents + Plugins)



MASTERING CLAUDE CODE CUSTOMIZATION

1. CLAUDE.md (The "Constitution")

- Purpose:** Acts as a behavioral contract that governs every session.
- Function:** It is a first file Claude reads. It stores project-specific "memories" like naming conventions, test commands and architectural rules to ensure the agent doesn't operate in a vacuum.
- Key Insight:** While a `README.md` is written to *inform* humans, a `CLAUDE.md` is written to govern agents.

2. skills/ (Knowledge & Automation)



- Purpose:** Reusable, task-matched workflows.
- Function:** Unlike slash commands which require manual typing, **Skills** use "semantic reasoning" to trigger automatically when Claude detects a task that matches the skill's description.
- Example:** A "deployment skill" might automatically run pre-flight checks and rollback scripts if it senses you are trying to push to production.

3. hooks/ (Deterministic Guardrails)



- Purpose:** Enforce security and quality with guaranteed outcomes.
- Function:** These are shell scripts or HTTP endpoints that run at specific lifecycle points (like `PreToolUse`).
- Guardrail Example:** A hook can intercept a `bash` command, see that it contains `rm -rf /`, and block it before it ever executes.

4. subagents/ (Isolated Delegation)

- Purpose:** Manage complex tasks without cluttering the main conversation.
- Function:** Subagents run in their own context windows with independent permissions.
- Usage:** You might spawn a Security Subagent that only has "read-only" access to scan code for vulnerabilities, keeping the main agent focused on feature development.

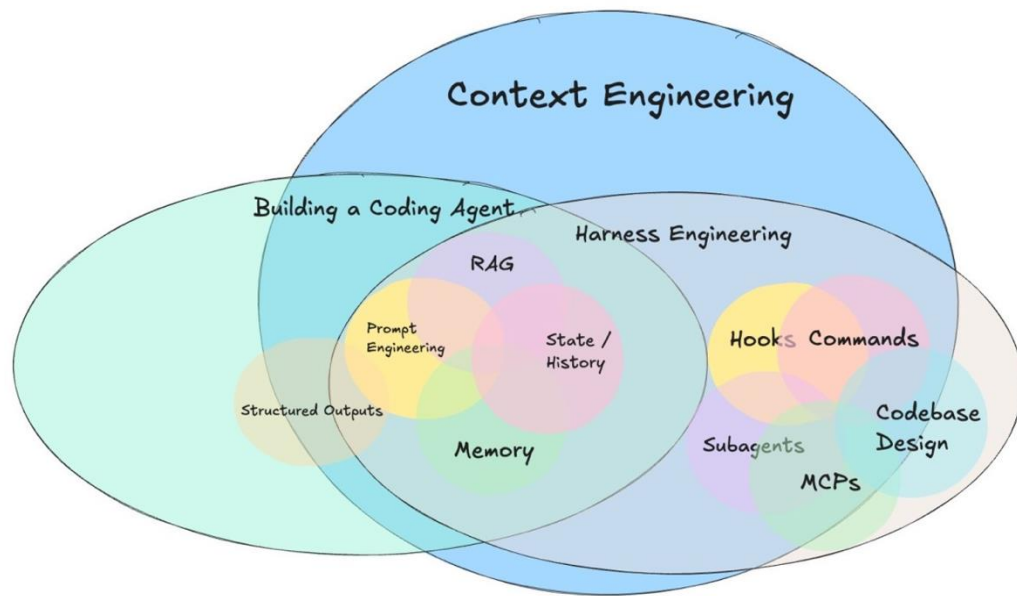
5. plugins/ (Distribution & Alignment)

- Purpose:** Bundling and team-wide synchronization.
- Function:** A plugin packages skills, hooks, and subagents into a single installable directory.
- Benefit:** This ensures every engineer on a team is using the same "aligned" setup, preventing "it works on my agent" inconsistencies.

Harness Engineering

Agent = Model + Harness

- Context engineering optimizes what an AI agent sees (data, prompt, input) to improve a single decision.
- Harness Engineering builds the infrastructure, feedback loops, and validation steps surrounding the model to ensure reliable performance over long-term, autonomous tasks.



EFFECTIVE HARNESSES FOR LONG-RUNNING AI AGENTS

INITIALIZER AGENT

1. CONTEXT SETUP:



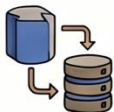
Sets environment & installs required dependencies (e.g., git, init.sh) for clean execution state.



2. STRUCTURED SPEC:



Generates comprehensive feature list (e.g., tests.json) with 200+ detailed requirements (start 'failing').



CODING AGENT

1. WORK INCREMENTALLY:



Focuses on ONE failing feature per context window (session) to prevent overloading.

2. GIT AS MEMORY:



Commits code & logic history into git to enable state tracking and easy reverts if blocked.

3. CLEAN STATE:



Ensures codebase is production-ready, documented, and bug-free before ending each shift.

CRITICAL PRACTICES

1. E2E VERIFICATION:



Mandatory full end-to-end testing (e.g., with Puppeteer) to confirm human-user experiences, not just unit tests.



2. PERIODIC PROGRESS

LOG: Updates specific progress files (e.g., claude-progress.txt) as an explicit hand-off artifact for the next session.



HARNESS DESIGN FOR LONG-RUNNING APPLICATION DEVELOPMENT

1. OVERCOMING 'NAIVE' AI LIMITATIONS



Context Anxiety & Coherence
(loss of goal tracking)



Self-Evaluation Bias
(overly positive grading)

2. THE MULTI-AGENT ARCHITECTURE (GAN-INSPIRED)



THE PLANNER
(Specification & Scope)



THE GENERATOR
(Feature implementation in 'Sprints')



THE EVALUATOR
(QA, skepticism, interacts with Playwright)

Professional Software
Development Lifecycle Mimic

3. KEY FINDINGS AND EVOLUTION



Subjectivity can be Graded
(turning taste into Design Quality & Craft)



The Power of Interaction
(live app interaction > static screenshots)



Harnesses must Evolve
(changes with improving models; QA vital for 'last mile')

CORE TAKEAWAY



Combine distinct specialized agents for a robust long-running harness.